0 General Introduction

Course Schedule:

32 class hours, 4 hours per week (Tuesday & Friday), Weeks 9–16, closed-book exam (60%), Homework (40%)

Main Course Content:

1. Consistent Hashing、Bloom Filters、Count-Min Sketch
2. How to describe sample similarity (similarity search, Jaccard Similarity, Cosine Distance, Norm, nearest neighbors, dimensionality reduction, MinHash)
3. Generalization and Regularization
4. Linear-Algebraic Techniques: PCA
5. Linear-Algebraic Techniques：SVD
6. Sampling and Estimation
7. Convolution

Main External Reference

http://web.stanford.edu/class/cs168/index.html

CS 168: The Modern Algorithmic Toolbox, Spring 2024

Supplementary Knowledge:

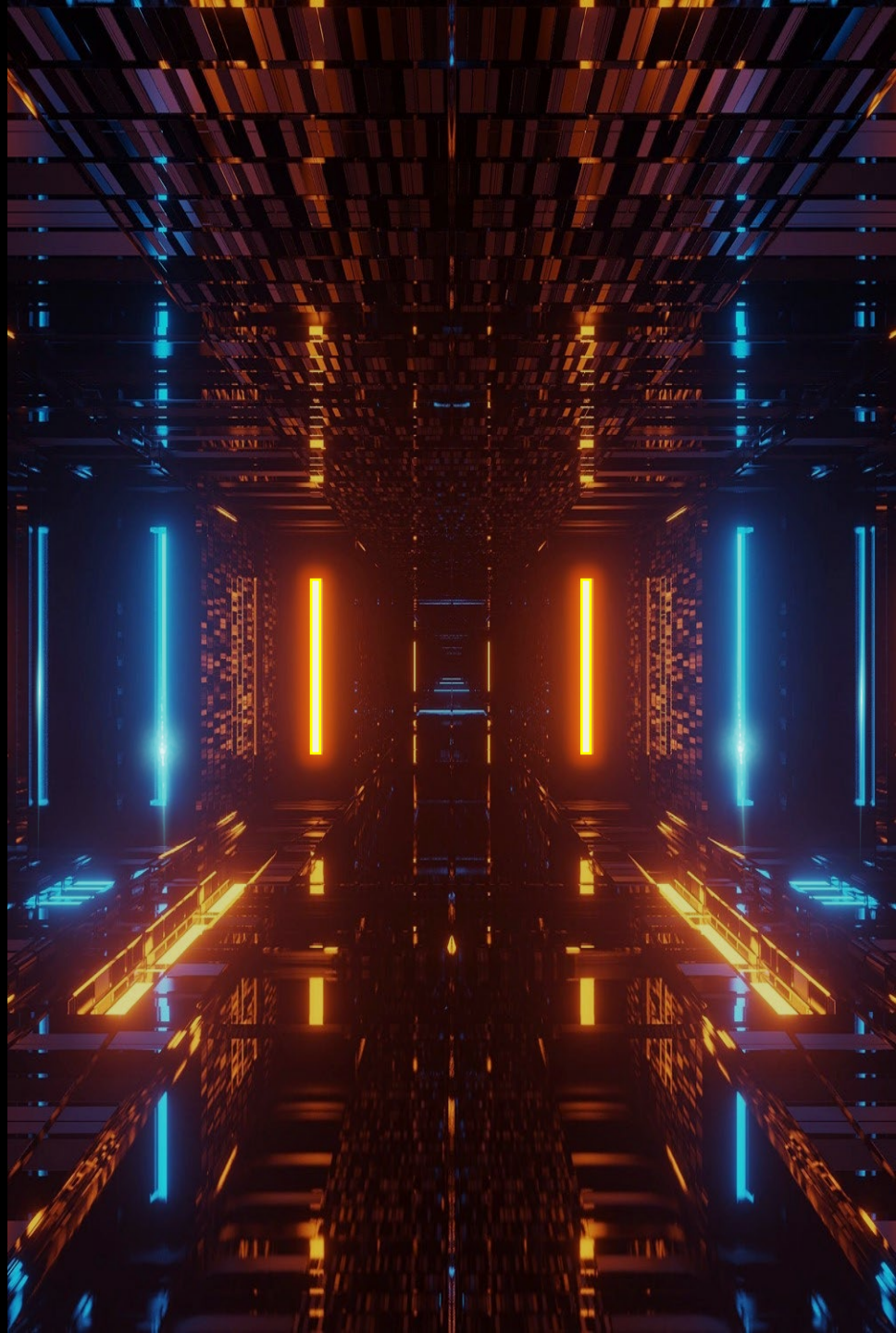
1. Sparse vector/matrix recovery (compressed sensing) and mathematical programming
2. Privacy-preserving computation

2026 04.28 ——→ 06.19

数据科学与大数据技术的数学基础

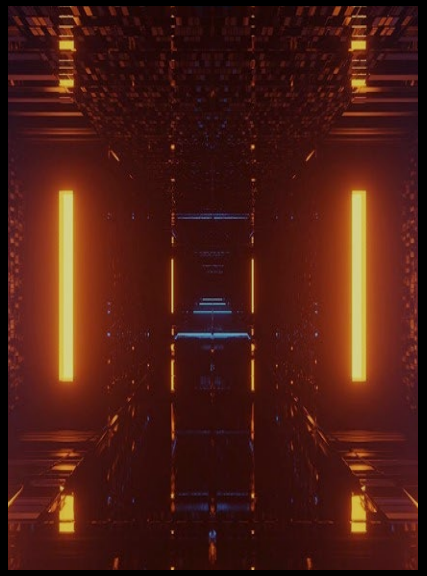
Mathematical Foundations of Data Science

罗霄
317834051@qq.com



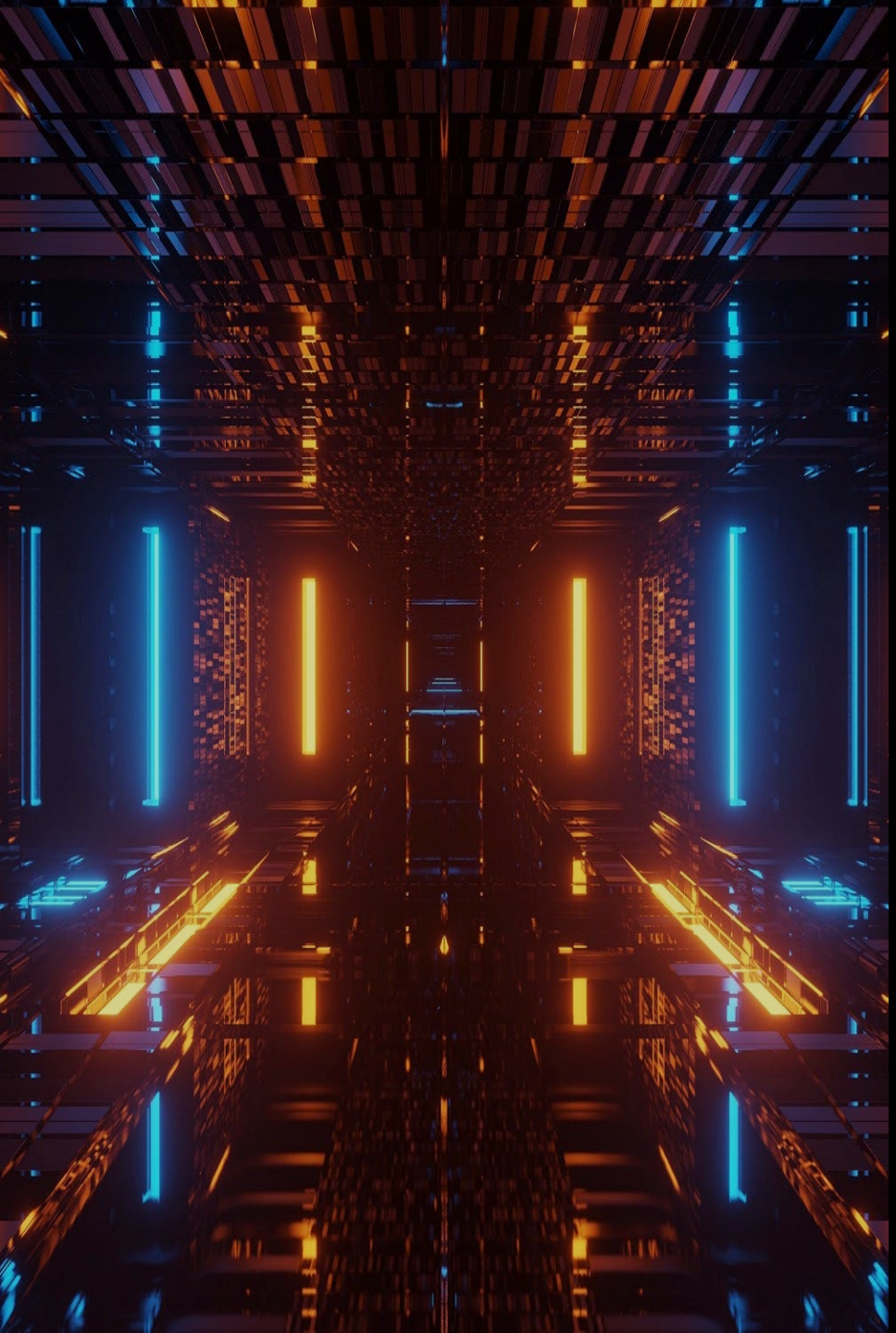
数据科学与大数据技术的数学基础

Mathematical Foundations of Data Science



/01 一致性哈希

Consistent Hashing



Contents

PS redis: sharding

Assumption

1.Sub-library rules: random allocation

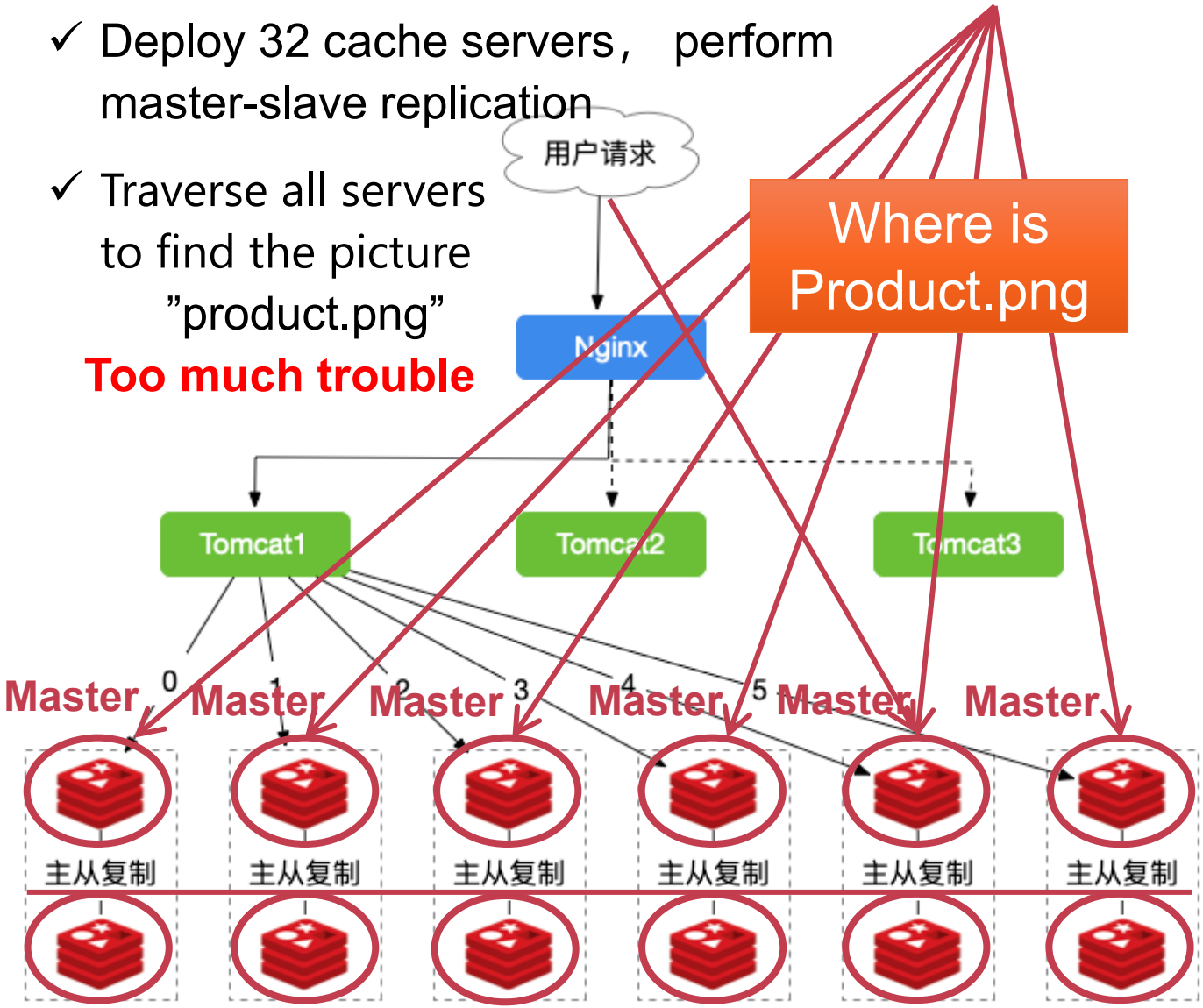
2.Stored value type: unordered hash table with key-value pairs

3.Key-value pairs involve operations: add, get, remove a single key-value pair, get all key-value pairs, check if a key exists

4.Final Hash task: store user information, user homepage visits, combined query (Internet)

- ✓ **No. Image8000w ÷ each server 500w = 16**
- ✓ Deploy 32 cache servers， perform master-slave replication
- ✓ Traverse all servers to find the picture ”product.png”

Too much trouble



PS redis cluster using Hash

Role of Hash

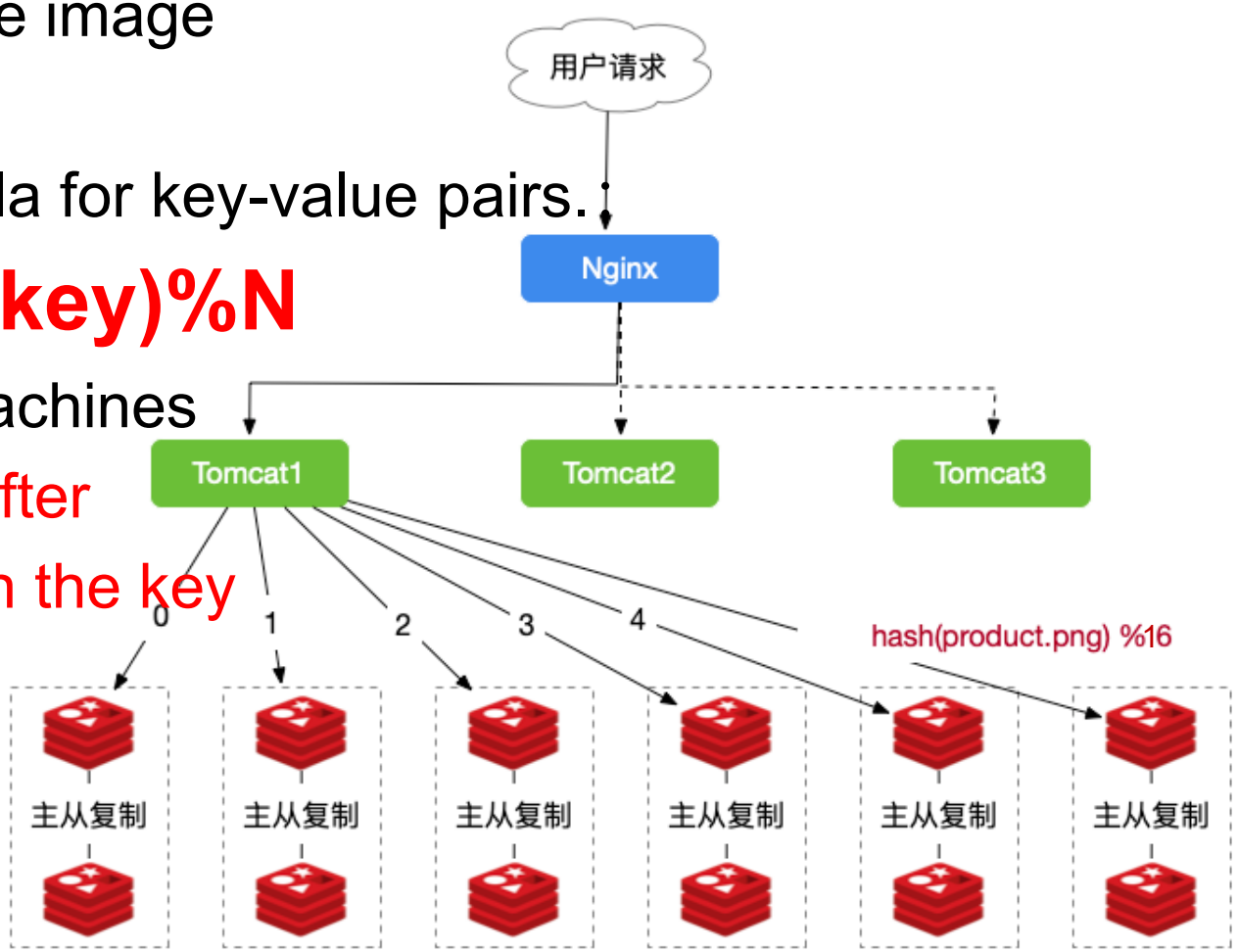
Using Hash, each image can be located to a specific server in Sub-library

1. **Goal:** Query the image "product.png“
2. Mapping formula for key-value pairs.:

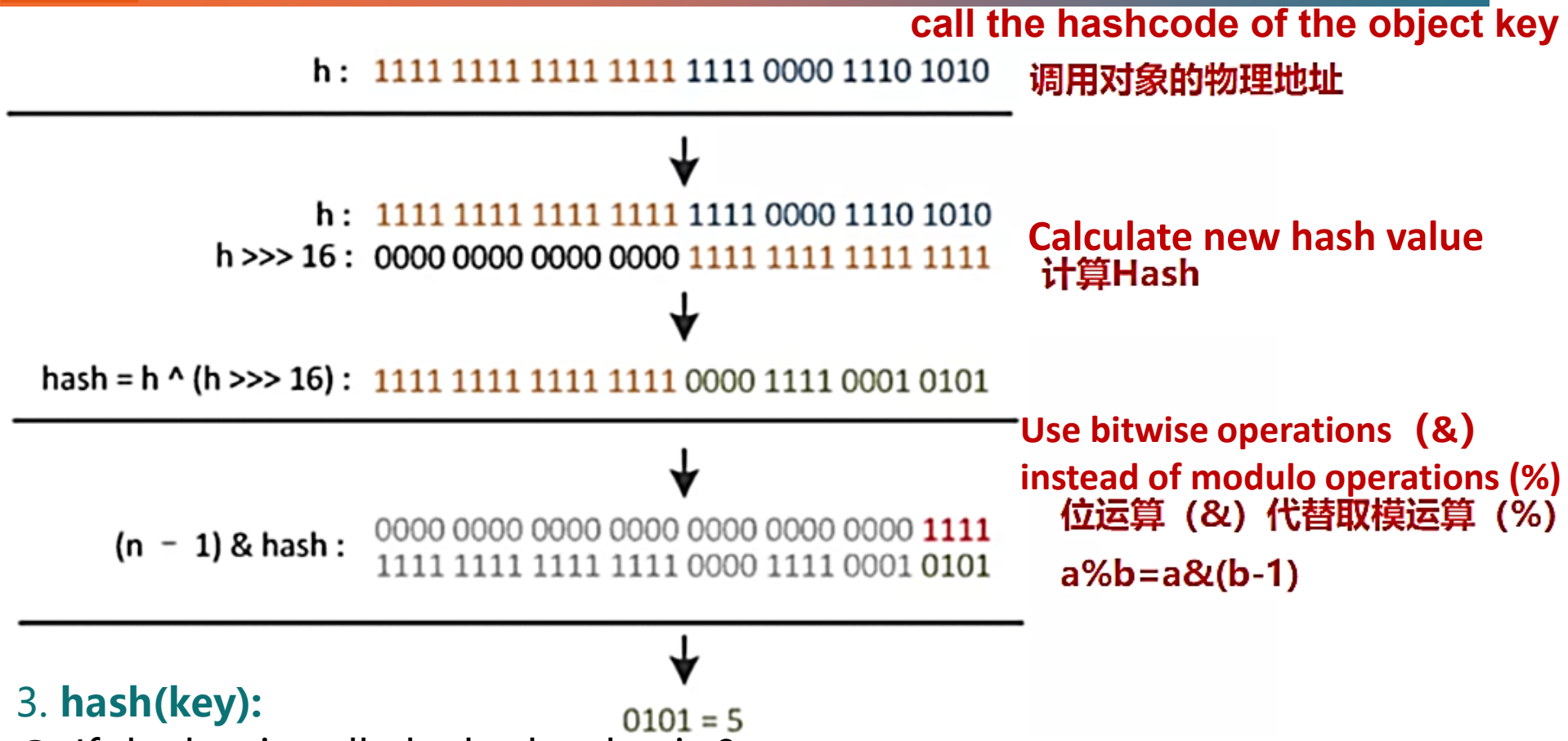
hash(key)%N

N:number of machines

take the **modulo** after **hash operation** on the key



PS redis cluster using Hash



3. **hash(key):**

① If the key is null, the hash value is 0.

② Otherwise, XOR the physical address value of the key with the physical address value right-shifted by 16 bits.

4. **modulo operation**

① %N means taking the previous step's result modulo the number of machines.

② When performing division on a negative integer in modulo operation, rounding is done toward negative infinity.

PS redis cluster using Hash

Role of Hash

Using Hash, each image can be located to a specific server in Sub-library

The time spent in traversing all servers is eliminated, thus greatly improving performance.

1. Objective: Query the image "product.png

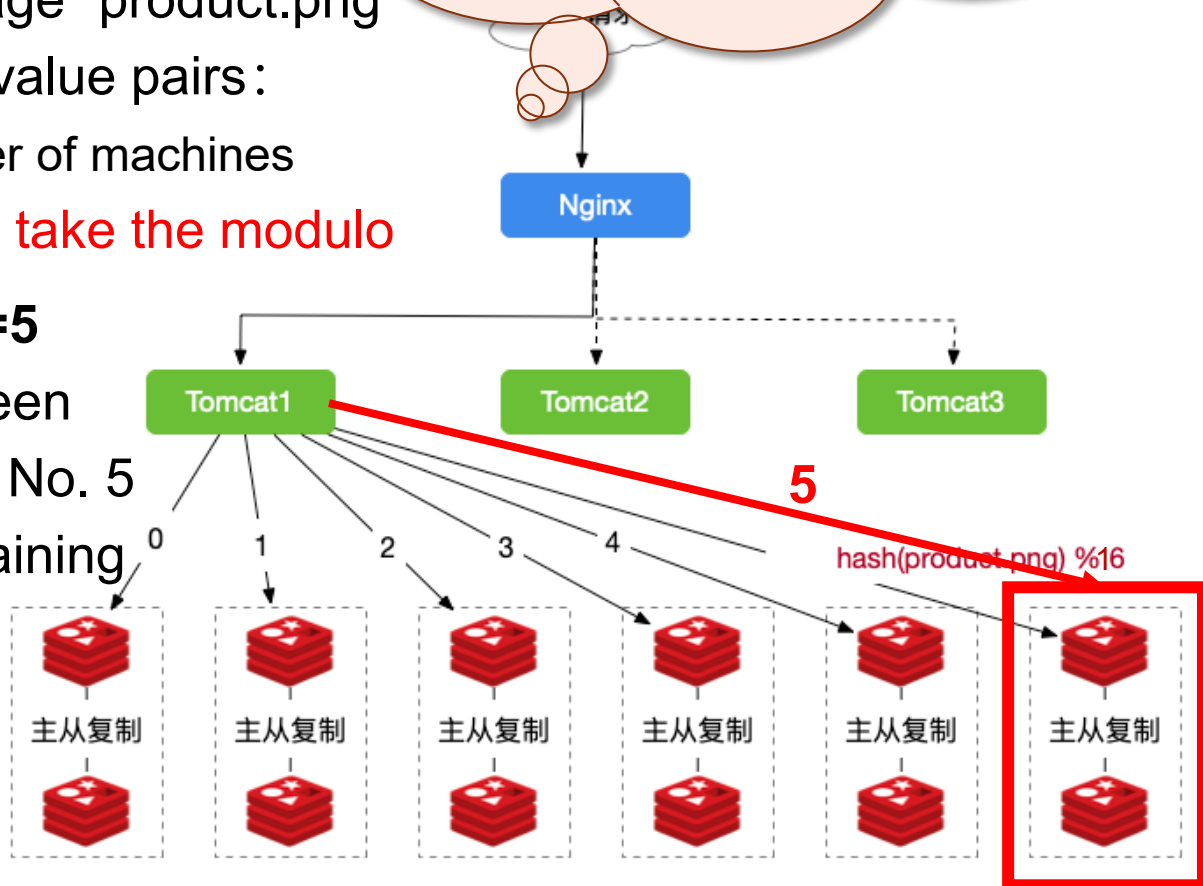
2. Mapping formula for key-value pairs:

hash(key)%N, N the number of machines

compute the hash (key) and take the modulo

3. hash(product.png)%16=5

The remainder is 5, the sixteen remainders are 0~15, Go to No. 5 of the machines by the remaining number



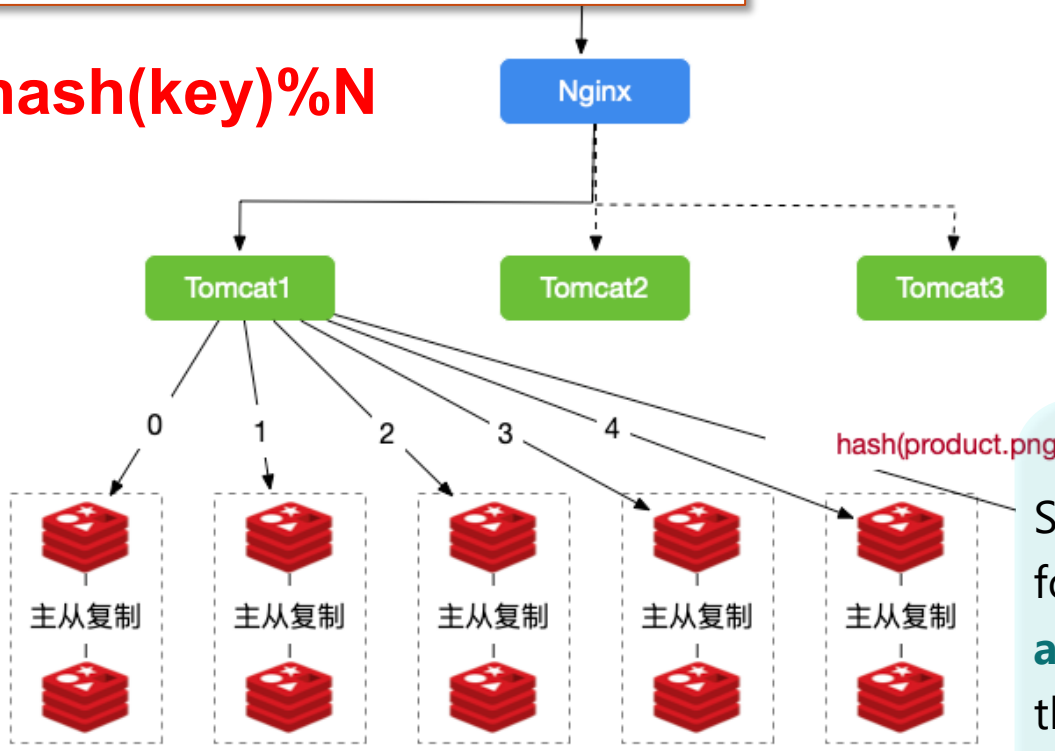
PS What are the problems when redis clusters using Hash?

Role of Hash

Using Hash, each image can be located to a specific server in Sub-library

h: 1111 1111 1111 1111
1111 0000 1110 1010

hash(key)%N



By moduloing hash, we don't need to iterate over all Redis servers → **performance improvement**

But when the Redis server changes, all cache locations will be changed

If the server fails:

Remove the failed machine and change the modulus from 8 to 5, where does "product.png" stored?

Conclusion

Since hash algorithm uses modulo for caching, **the Hash Consistency algorithm was born** to circumvent the above situation

1.2 Consistent Hashing

1. Consistent hashing need modulo?

A： Yes

2. Isn't it the same as hash?

A: No.

Hash is modulo the **number of servers (%N);**

consistent hash is modulo 2^{32}

Originally proposed by **MIT in 1997** to solve the hot spot (hotspot) problem in the Internet, now it has been widely used **for load problems in distributed applications.**

The result of **hash** will **change as the number of servers change**

The result of **consistent hashing** **don't vary with the number of servers**

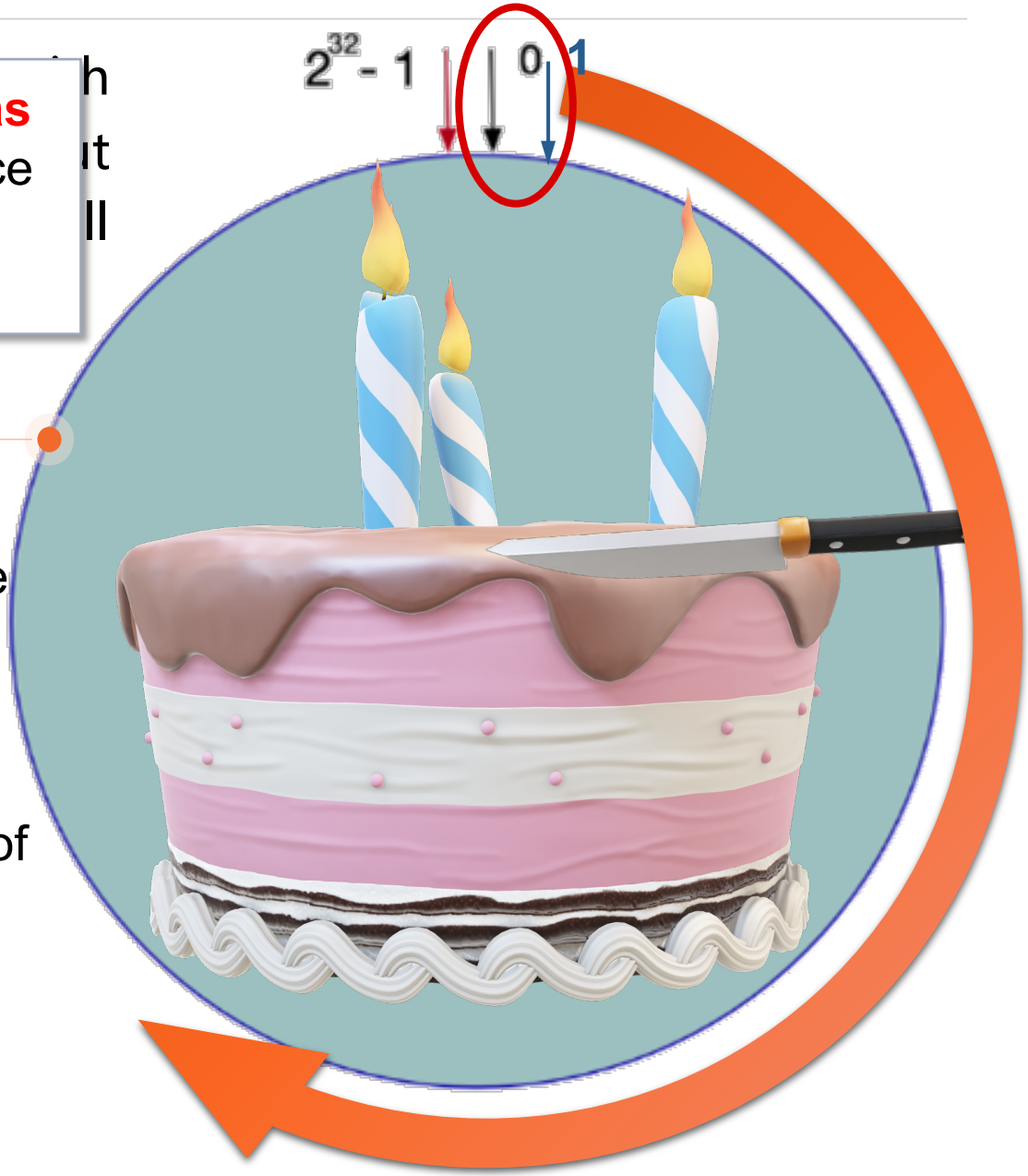
字服务器……很难吧，是不是画到草稿纸外头了还没画完还没画完还没画完还没画完

1.2 Consistent Hashing: 1. Building a hash ring

Consistent Hashing

Considering the **Hash space as a virtual circle**, the value space of Hash function is $0 \sim 2^{32}-1$ (a 32-bit unsigned integer)

Hash ring

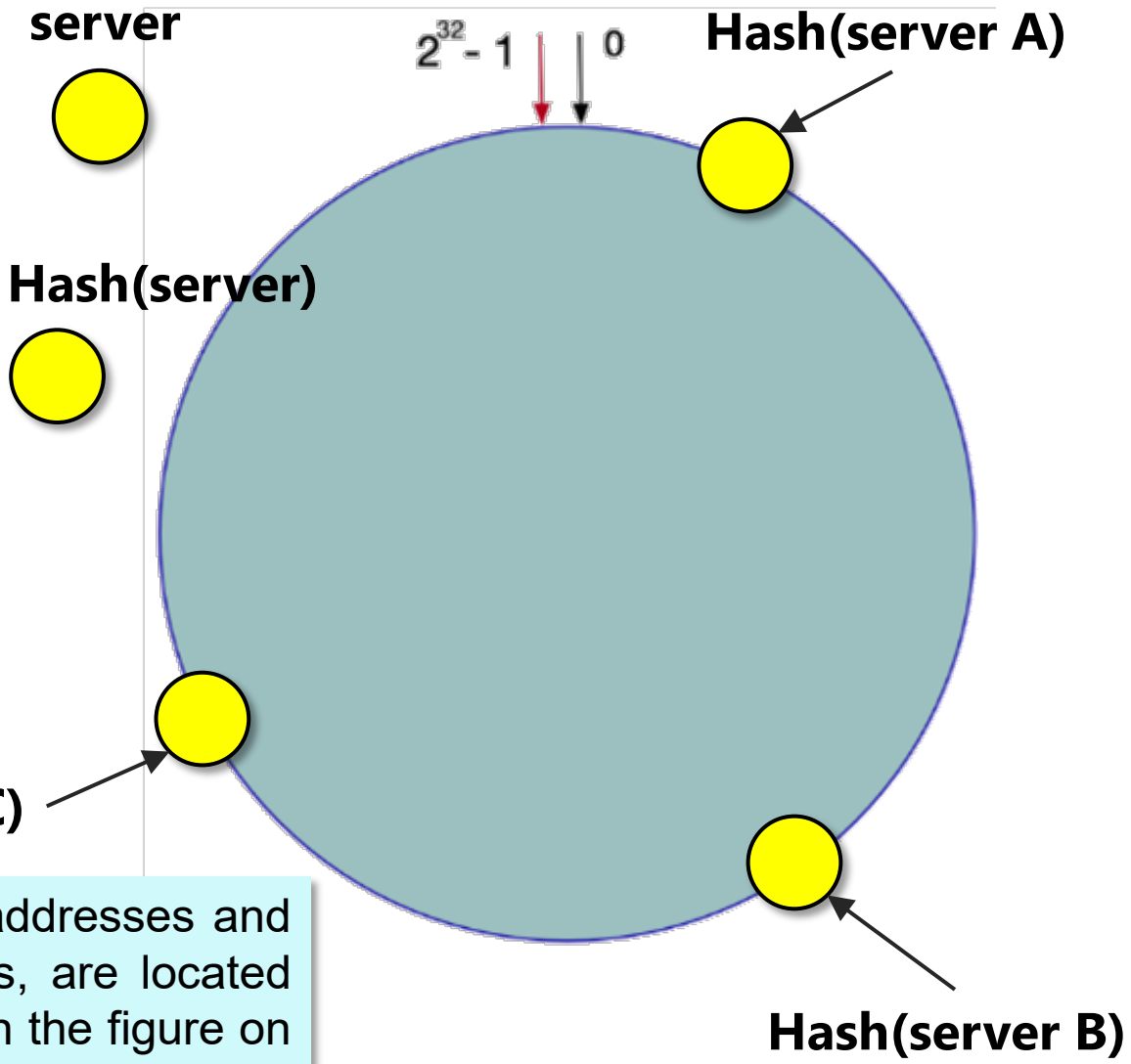


- Clockwise direction is positive
  - The point directly above the circle represents **0**
    - The first dot to the right of the 0 point represents **1**
      - and so on

1.2 Consistent Hashing: 2. Confirm server location

Consistent Hashing

- Select the **IP or host name** of the server as the **keyword key**
- Perform a **hash operation** on the server's key separately
- Determine the **location of each server on the hash ring**

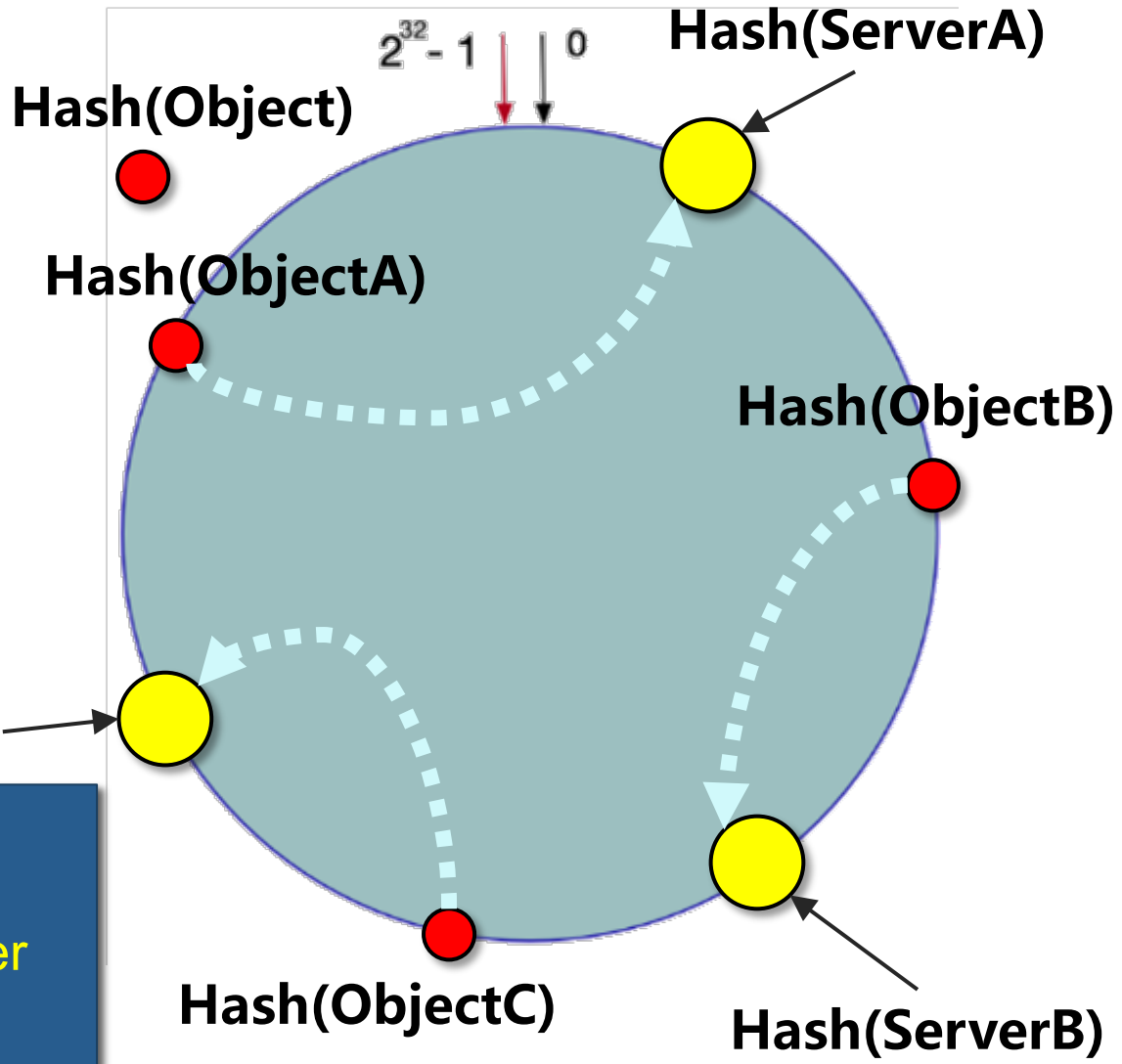


3 machines, after using IP addresses and performing hash calculations, are located in the ring space as shown in the figure on the right

1.2 Consistent Hashing: 3. Confirmation of data storage location

Consistent Hashing Determination of the position of the data object on the ring

- Use the same function to calculate the hash value of the key
- Locating data on a hash ring
- Search clockwise and deposit the first server encountered

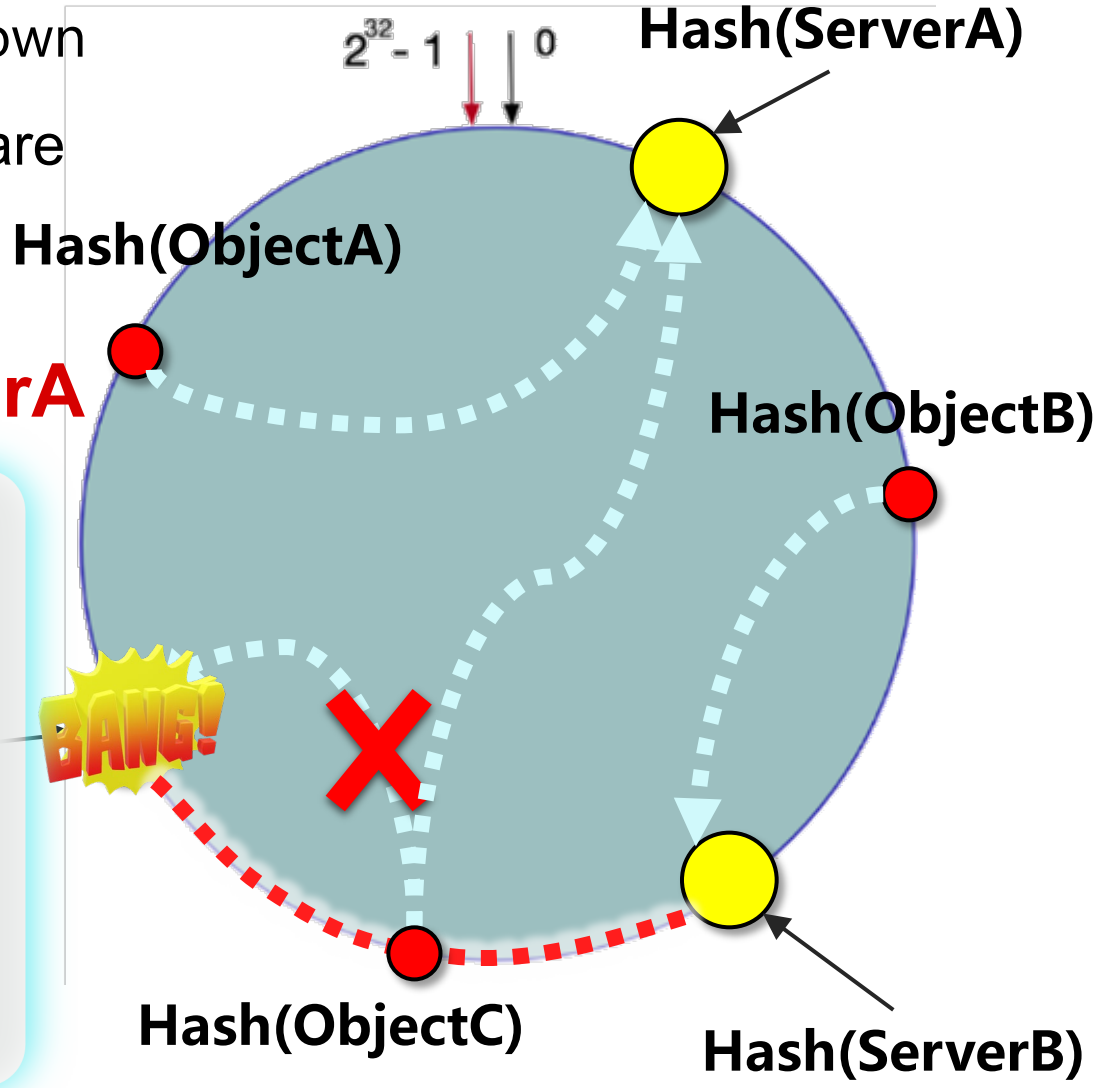


Get data:
Calculating hash(Object);
Determine the closest server in clockwise direction;
Gets the data.

1) In case of data server failure

- Suppose server C is down
- ObjectA and ObjectB are not affected, ObjectC needs to be relocated

To serverA

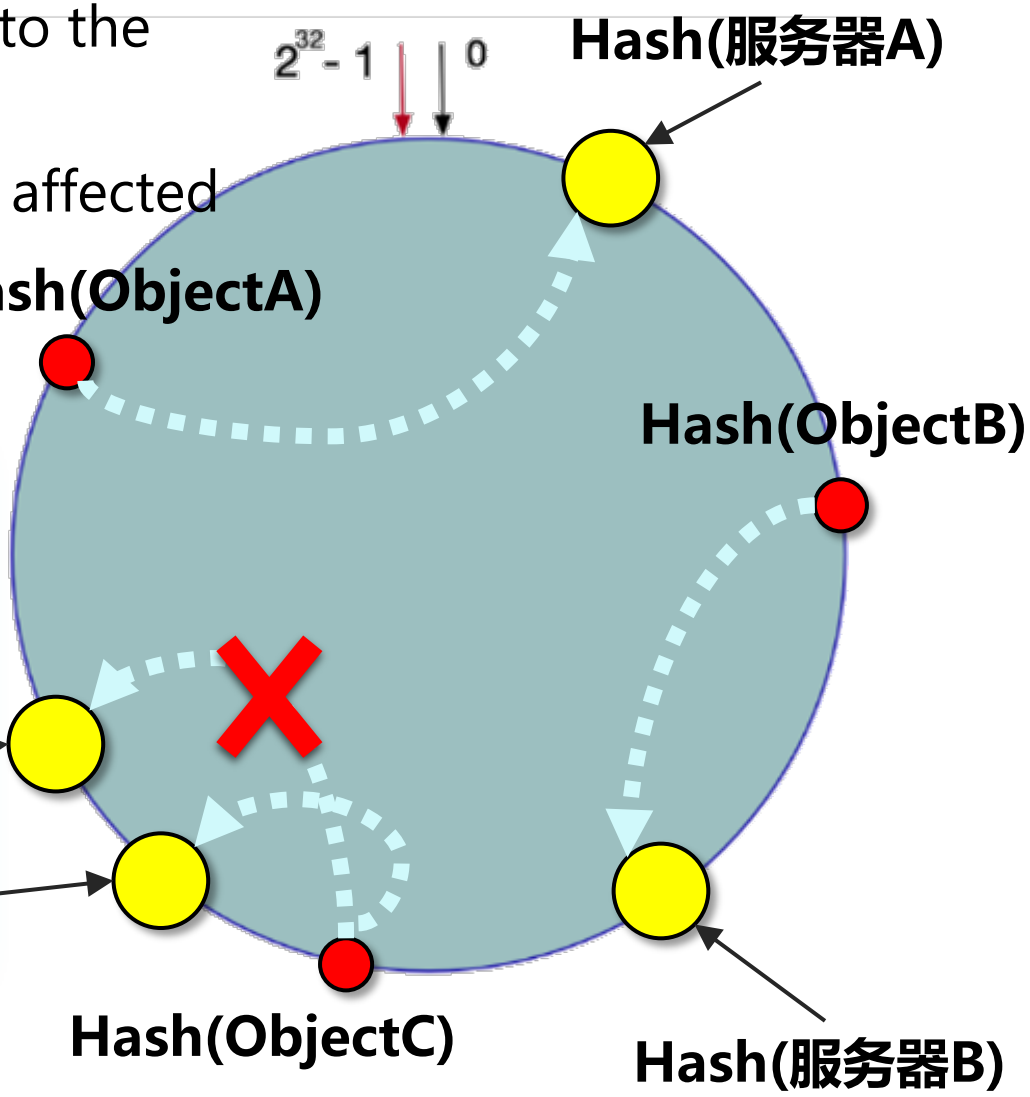


In Consistent Hashing, if a server is unavailable, **the only data affected is between this server and the previous server in its ring space** (in this case, between C and B), and the rest is unaffected

2) Server X is added to the system

- Suppose a server X is added to the system
- ObjectA and ObjectB are not affected
- ObjectC repositioning

In Consistent Hashing, **only a small portion of the data in the ring space needs relocating when nodes are added or removed**. It's very fault tolerant and scalable.

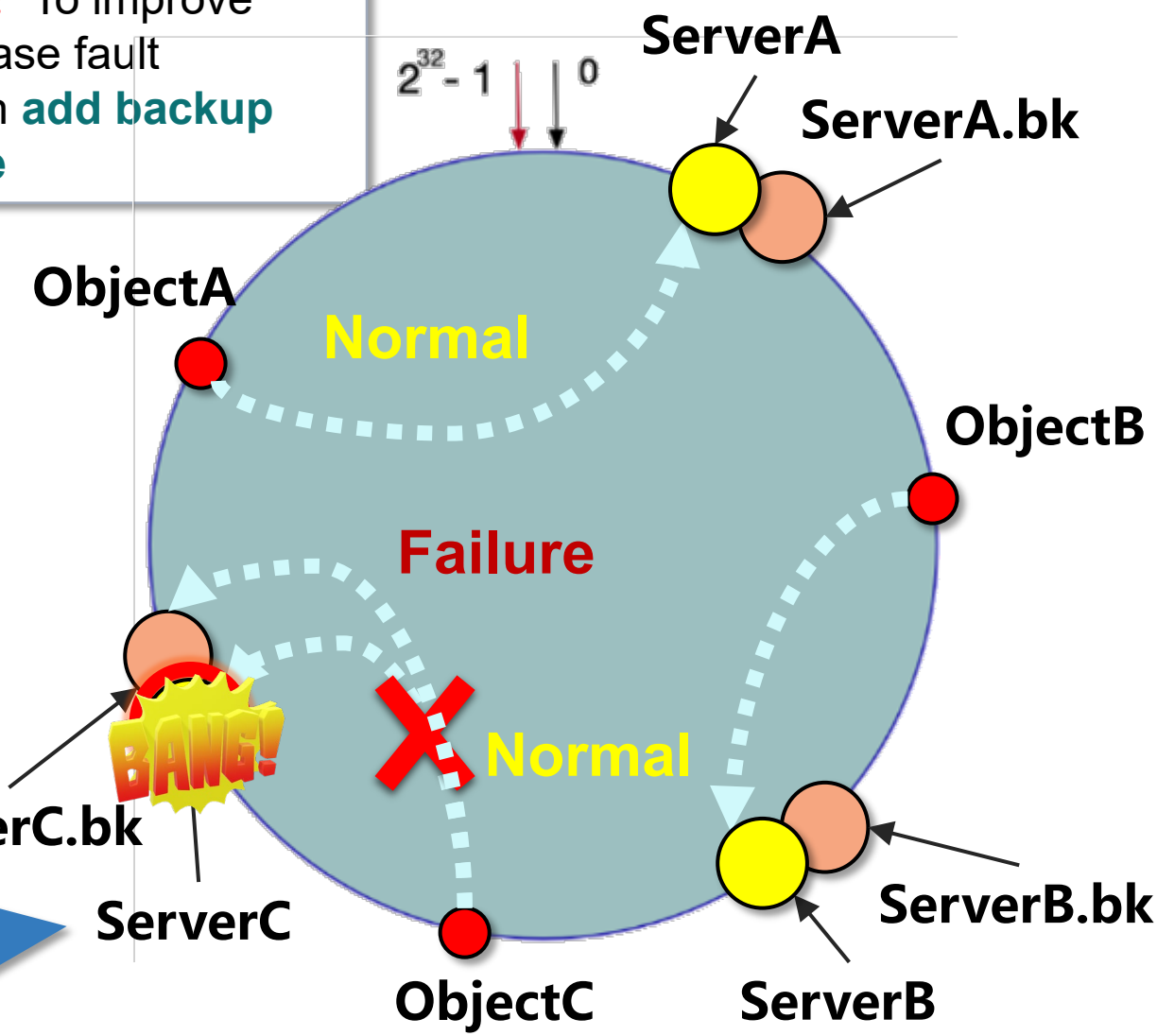


1.3 Consistent Hashing Optimization and Improvement: Increase Fault Tolerance

Can you combine consistent hashing with Master-slave together? To improve consistent hashing and increase fault tolerance for failures, you can **add backup nodes for each server node**

- Master and Slave data server **keep data synchronized** ;
- In case of failure, the algorithm **switches to the slave server for data** ;

Server C acts only as a data storage node and not also as a slave node for neighboring nodes



1.3 Consistent Hashing Optimization and Improvement: Increase Fault Tolerance

Specific handling process in case of server C failure

1.Directs ObjectC to ServerC according to consistent hash;

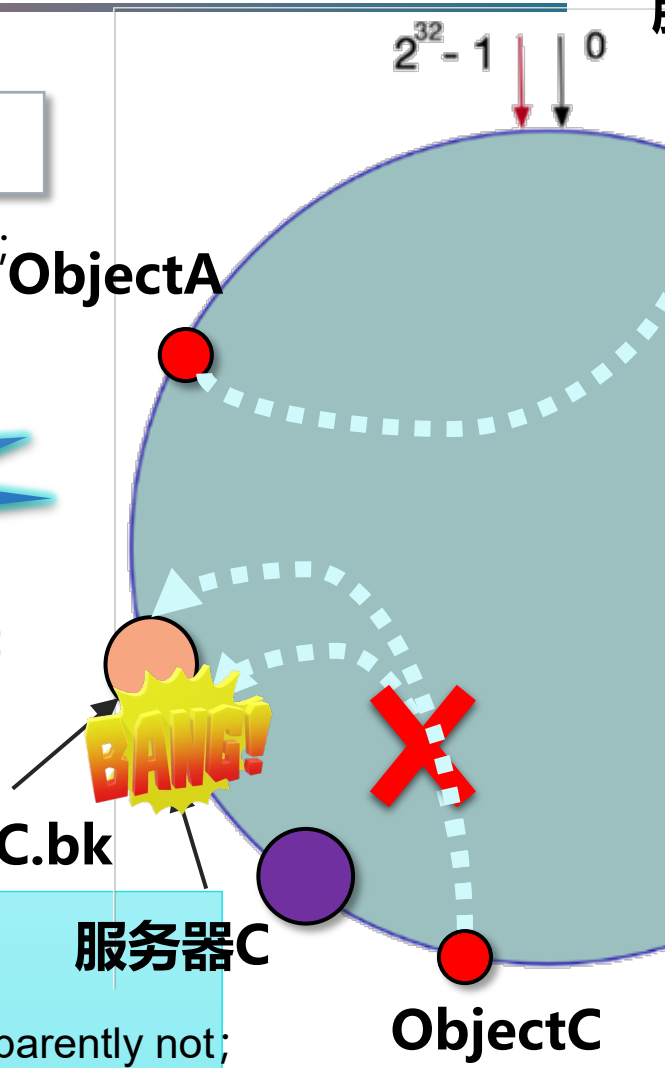
~~Server C failure can neither write nor read~~ ;

2. Find the backup(slave) node C.bk of server C to write or read ;

3. If Server C **recovers** ;

Based on data synchronization, it will **get the latest information from the backup(slave) node**

Master-Slave Replication



Q: If add a new data server X in front of server C, how does it work ?

Write

New data is written directly to the new server

Read

1. ObjectC first find the new server X for data, apparently not;
2. Continue through clockwise direction until data is found on server C, grab the data and run (not copy just cut) ;
3. Write the data to their own clockwise new server X

1.3 Consistent Hashing Optimization and Improvement: Data Skewing Problem

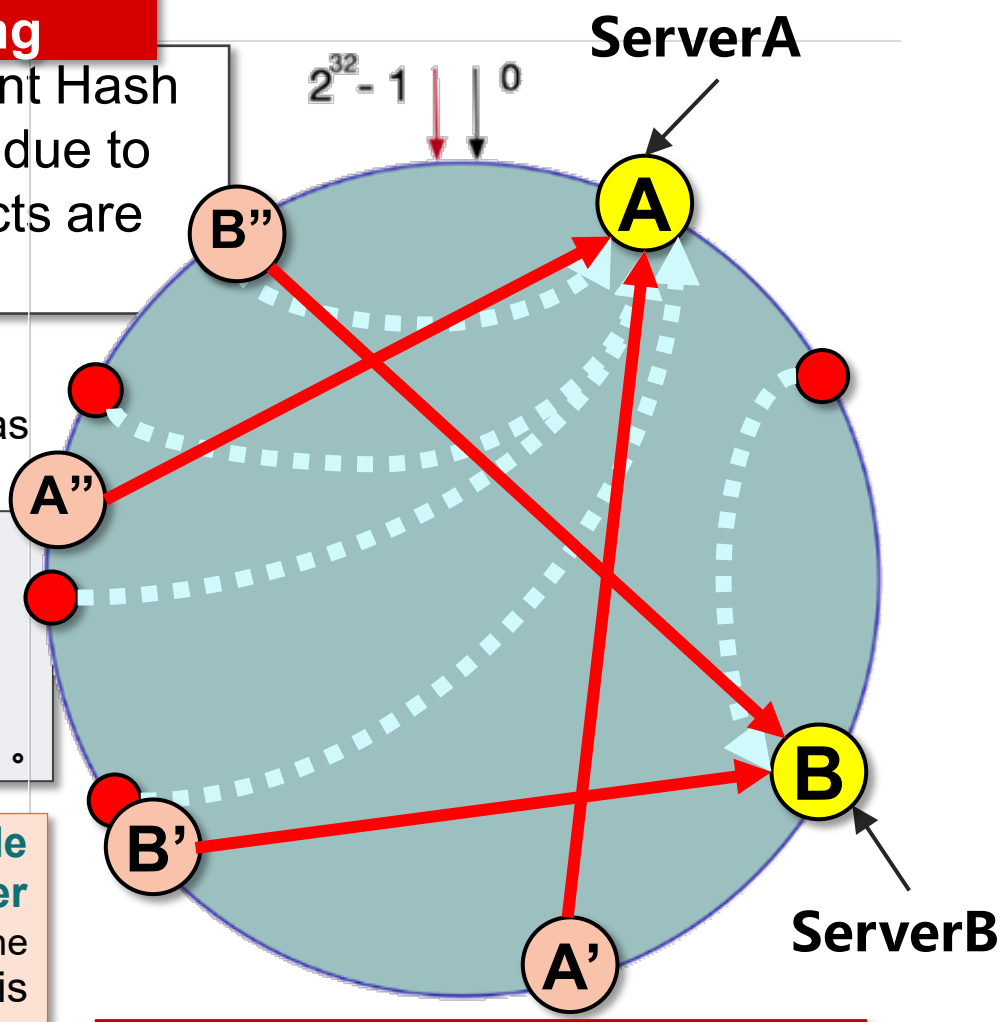
Assumptions: uneven distribution of cluster servers on the ring

In the case of too few consistent Hash nodes, it is easy to **skew the data** due to uneven distribution of nodes (objects are mostly cached on one server)

1. when calculating the key of the server, make the key values as evenly distributed as possible;

2. **Virtualization** is used to **isolate a physical server into multiple virtual machines**, adding multiple server nodes to maintain a balanced distribution in the form。

Virtual Node Mechanism: **Multiple hashes are computed for each server node**(**New code:** adding a number after the server IP or host name), and a service node is placed at each computed result location, called a virtual node.
The number of virtual nodes is proportional to the capacity of the server.



Only add one additional step: map virtual nodes to actual(physical) points

1.4 Computational loss of consistent hashing: expected load

M objects need to be stored in N cache servers, **each with an expected load of M/N** according to symmetry

Q1: data skew, data distribution
A: Virtual node mechanism (previous PPT)

Q2: Suppose we add a new cache server, which objects need to be moved?
A: In the expected case, adding the nth cache will **only result in M/N object relocation** (evenly distributed, PPT p27)
In contrast, **only M/N objects** on average in the **hash** method **do not need to be relocated** (%N, p17)

Adding new cache server:
The **new cache server can identify its "successors"** and send requests to the objects located in its scope of responsibility

Q1 : How can we quickly insert data into the corresponding in hash ring? How do we implement "Lookup " and "Insert"?
A : Given an **object x**, both operations can be attributed to effectively implementing a **rightward, clockwise scan** of the cache server s such that **h(s) takes the minimum value when h(s) ≥ h(x)**.
Q2: Can consistent hashing completely **prevent hash conflicts? Is the hashcode unique?**

1.4 Hash conflicts

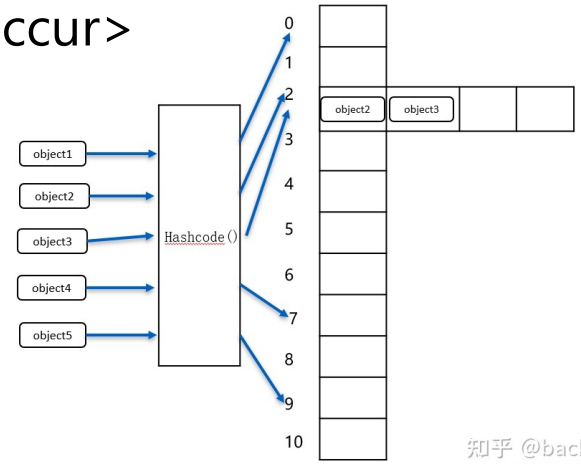
The goal of the hash function is to generate different hash values for all different objects

But it is unavoidable that different objects will generate the same hash value

Hash conflict

So different objects will stored in the same location in the container.

<Even if we use a **hash ring (hash table rolled into a ring)** up to the length of 2^{32}, as long as there are enough graphs to be stored, it is still possible for **different graphs to have the same fingerprint** (Or the same result is obtained by modulo calculation, and also hash conflicts will occur>



In Hashmap, this building action is divided into two main schools of thought, **chained tables and red-black trees** (链表与红黑树) .

1.4 Hash conflicts (in HashMap)

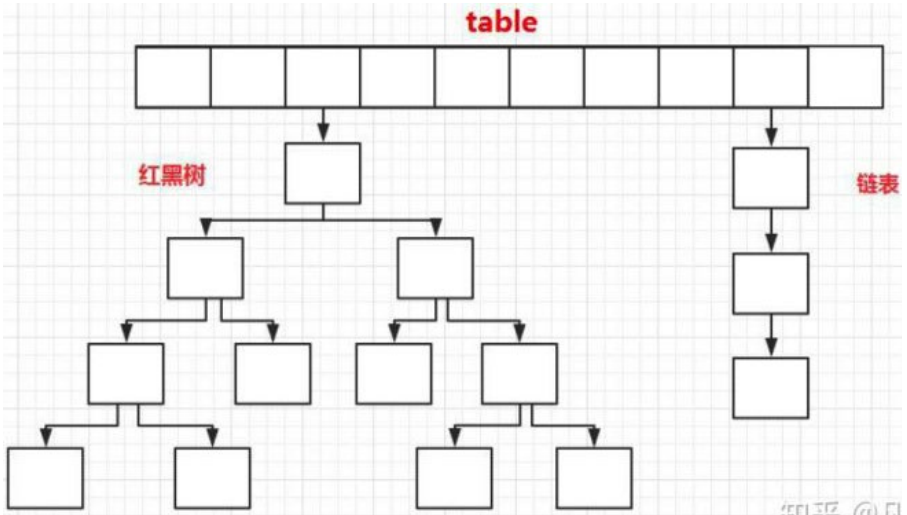
HashMap will determine whether the current position is empty, if there are already elements in existence, this element will be placed as the head node, using the **tail insertion method** to place the new element as the next node (can be understood as the **new arrival at the end of the queue, do not insert the queue**)

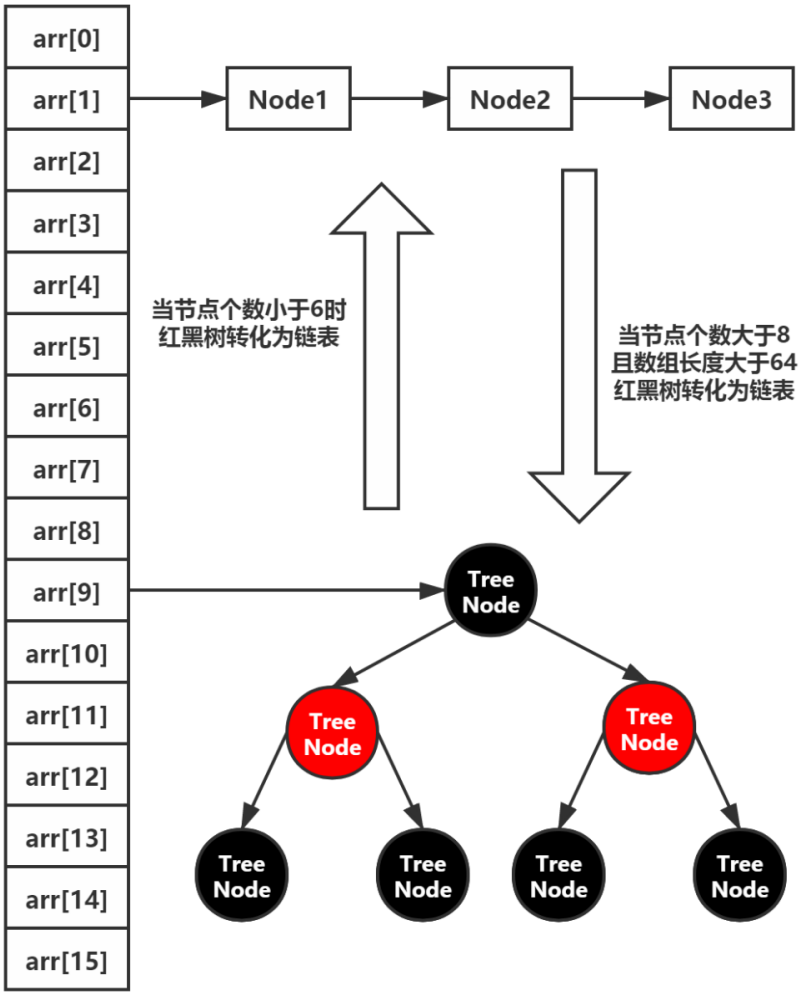
6. The ones queued at that address form a **chain table**

7. So theoretically, the **time complexity** of a lookup in an (unordered) linked table is $O(n)$

8. This kind of unordered queuing certainly cannot be applied to a server to store a large number of Objects, so when the chain table is too long, red-black trees are used.

What about the time complexity of the red-black tree?





1.4 Computational loss of consistent hash: expected load

Q2: What kind of data structure? To reach fast enough?

Therefore, we need a data structure for storing the cache name, keyed by the response hash, to support fast inheritance operations.

- **Hash table** is not good enough (no sequential information);
- **Heap** is not good enough (only partial order relations);
- **Binary search tree** maintain the full order of the stored elements. Since the running time of this operation is linearly related to the depth of the tree, it is a good idea to use a balanced binary search tree, such as a **red-black tree**.

Q3: binary search trees, red-black trees, balancing?

Ranking

Binary search tree

The value of each node in the left subtree is less than that node;
The value of each node in the right subtree is bigger.

The left/right subtree of binary search tree maybe **different lengths, imbalance**

Self-balancing binary search tree

Red-Black Trees

The longest path of a red-black tree from the root to the leaves is no more than twice the shortest path;
Use color change and rotation

Red-Black Tree pursues a rough balance, also think about the ease of operation

PS Red-Black Trees

Characteristics

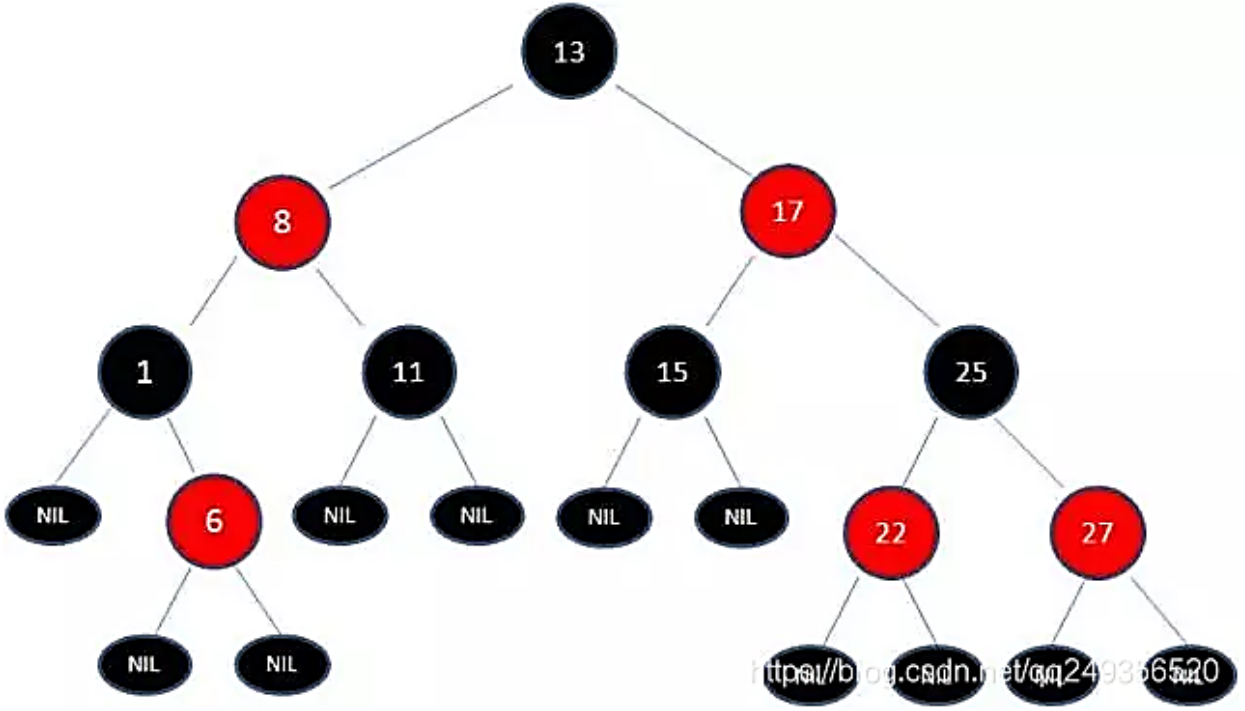
1)Nodes are red or black ;

2)Root node is black;

3)Each leaf node is a black null node (NIL node) ;

4)2 children of each red node are black (No two consecutive red nodes)

5)All paths from any node to each of its leaves contain the same number of black nodes



1. Look at the follow figure, now ready to insert node **21**

2. 21 is smaller than 22, so insert the left subtree of node 22

3. Since 22 is a red node, because (4), should be adjusted

4. Try color change first

PS Red-Black Trees

Characteristics

1)Nodes are red or black ;

2)Root node is black;

3)Each leaf node is a black null node (NIL node) ;

4)2 children of each red node are black (No two consecutive red nodes)

5)All paths from any node to each of its leaves contain the same number of black nodes

5. Change color can't solve the problem, try to use rotate

（1）左旋 对X结点左旋，即以X的右孩子Y为轴，将X结点转下来，变为Y的左孩子，简单记为左旋即把该结点变为左孩子

（2）右旋 对X结点右旋，即以X的左孩子Y为轴，将X结点转下来，变为Y的右孩子，简单记为右旋即把该结点变为右孩子

6. After rotating 2 times, change color according to the rules

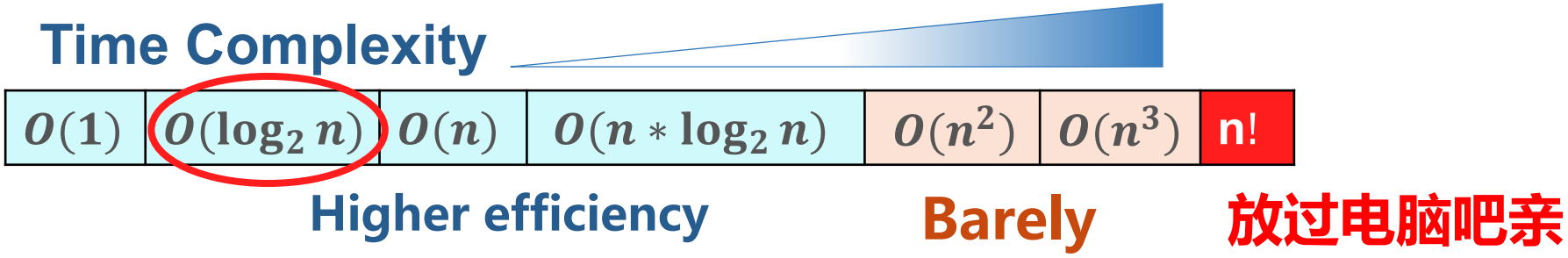
***The red-black tree gives up the pursuit of complete balance and pursues approximate balance, which is much simpler to achieve with a time complexity not much different from that of a balanced binary tree, ensuring that each insertion requires at most three rotations to reach balance.**

PS Time Complexity

Q4: What is the time complexity? Is the time complexity of find, insert and delete the same? Why use red and black trees?

Time Complexity

- $O(1)$: target element can be obtained **directly** in a single operation (e.g. dictionary or hash table)
- $O(n)$: n elements are first checked to search for the target
- $O(log\, n)$?



Dichotomous search Time complexity is $O(log\, n)$<extreme case>

PS Why the time complexity limit of dichotomous search is $O(\log n)$

An ordered array with 16 elements as follow. Please find 13

1	3	5	8	12	13	15	16	18	20	22	30	40	50	55	67
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

1. Choose the middle element as the center point, 13 is smaller than the center point, so only the first half of the array is considered

2. Repeat the process, each time looking for the middle element of the subarray

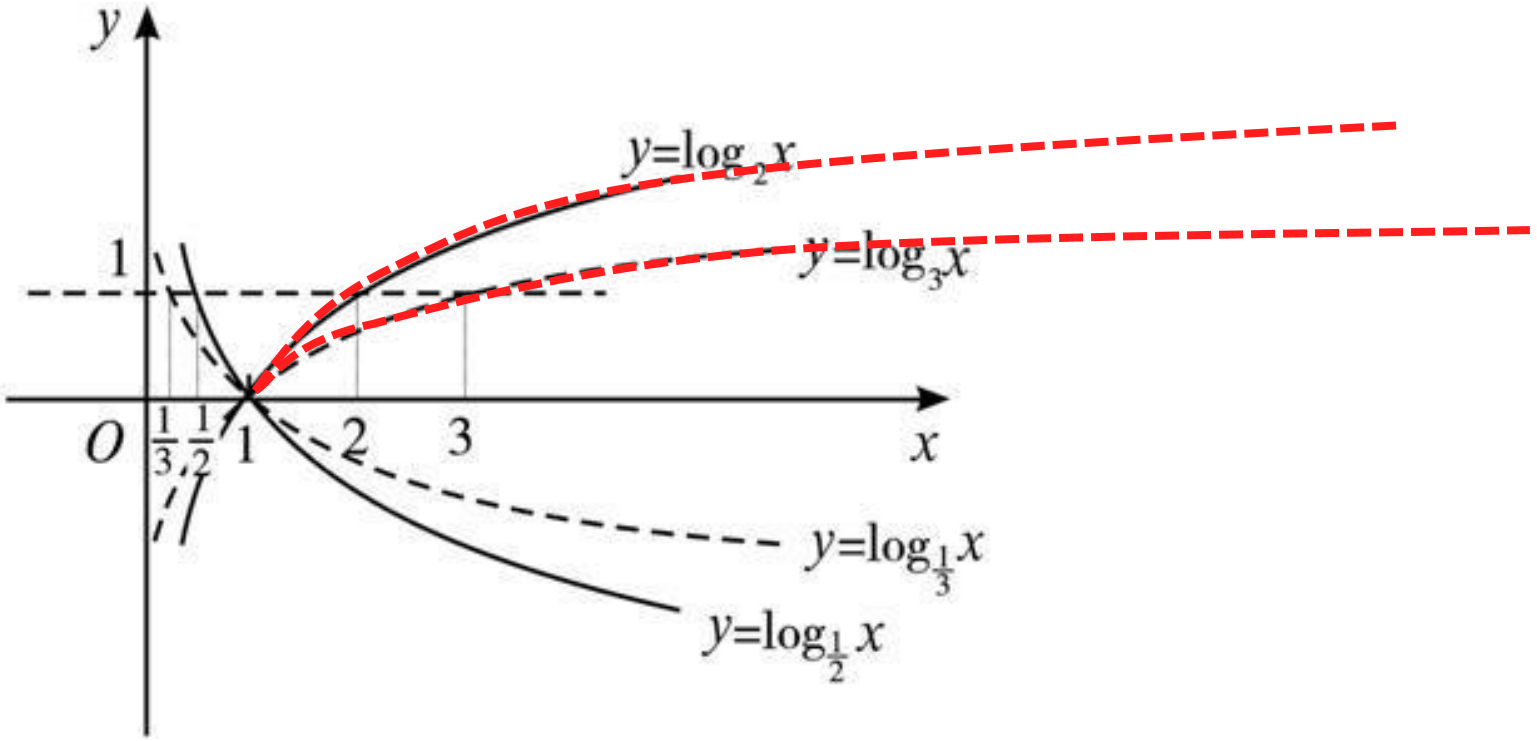
3. Each comparison with the middle element halves the search range. So in order to find the target element from the 16 elements, we need to split the array 4 times equally， i.e.

4. Similarly, if there are n elements

5. Multiply both sides of the equation by 2^k

PS

If Time complexity $O(\log_x n)$, does it matter what the base x is ?

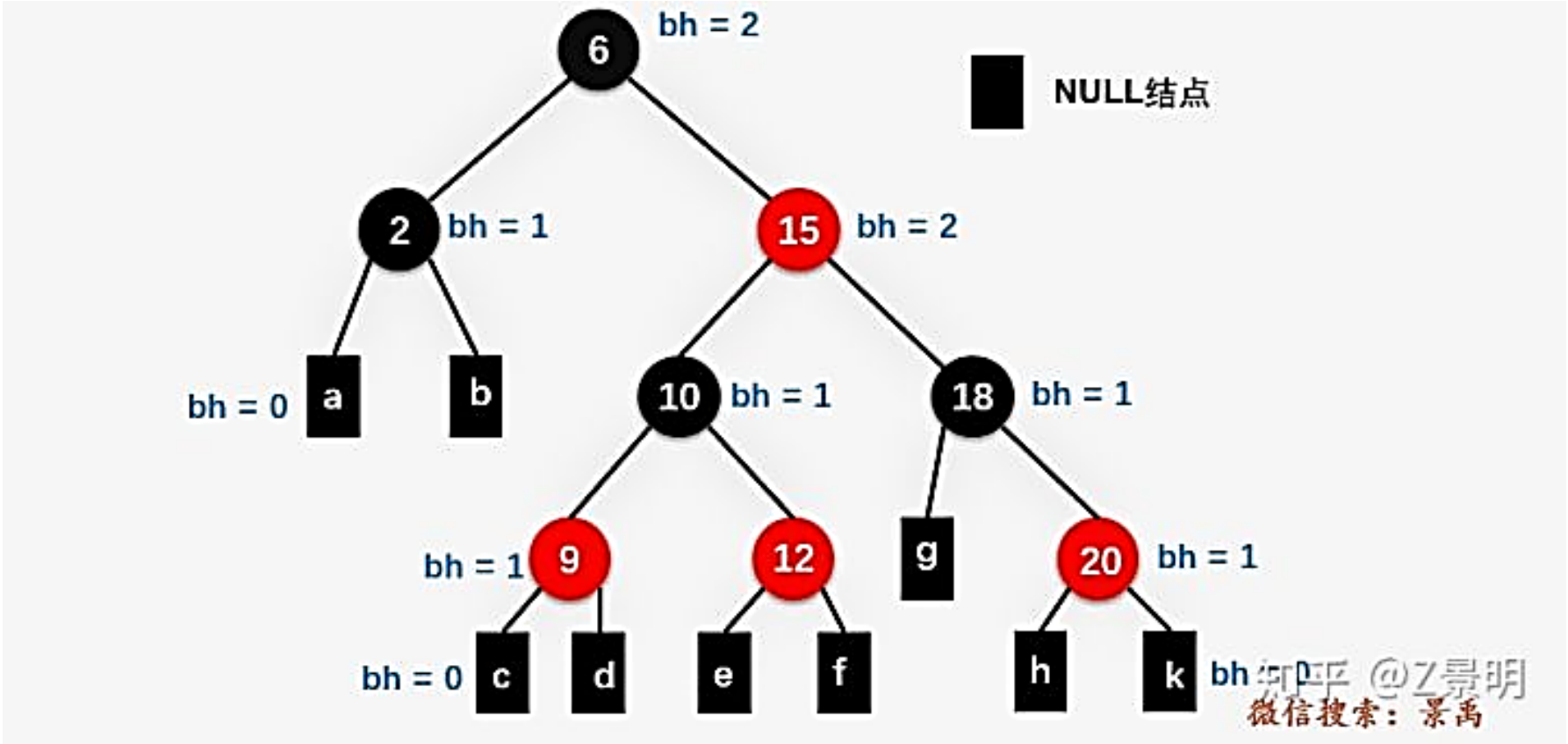


- ✓ As n increases, the curve tends to smooth out and the difference does not look particularly large
- ✓ If the **multiplicative relationship** between the different time complexities is **constant**, they could be approximated as **the same order of magnitude of time complexity**
- ✓ **Compared to the data size n, the bottom number is not important for studying the efficiency of program operation**

PS The approximate balance of the red-black trees

2) What is the black height of a red-black tree ?

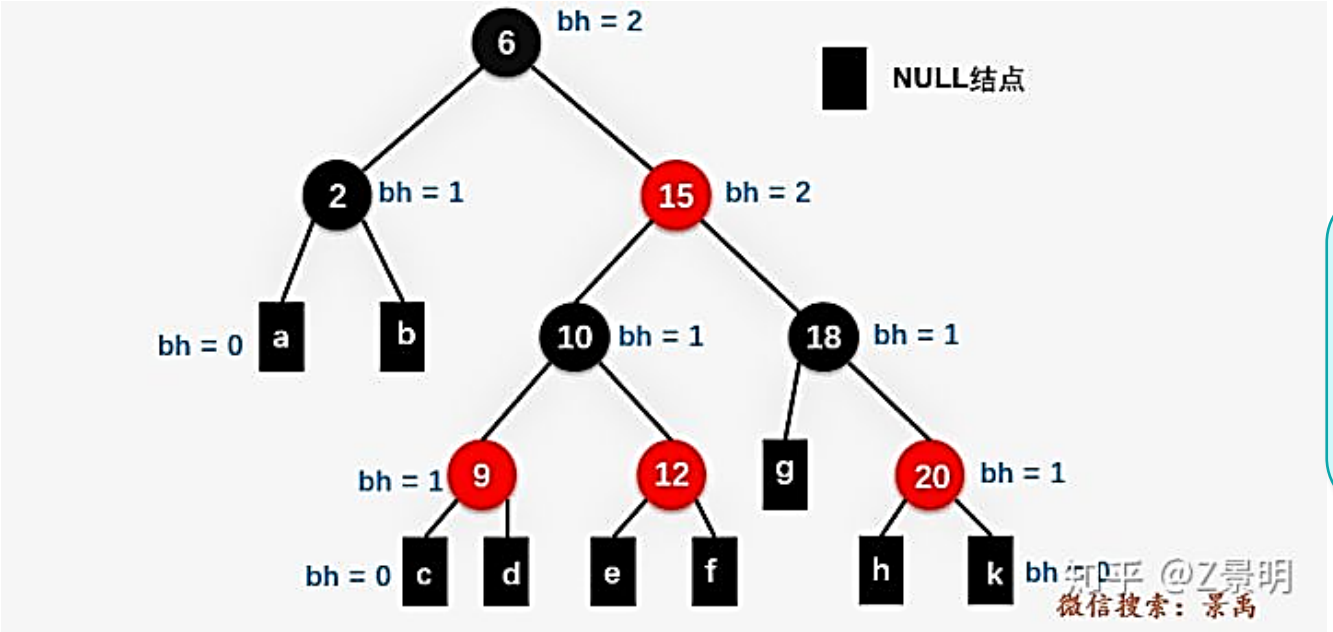
In a **red-black tree**, the number of black nodes contained on **any simple path from the root node x (without that node) to a leaf node** is called the **black height**, denoted $bh(x)$.



PS The approximate balance of the red-black trees

2) What is the black height of a red-black tree ?

In a **red-black tree**, the number of black nodes contained on **any simple path from the root node x (without that node) to a leaf node** is called the **black height**, denoted $bh(x)$.



In red-black tree

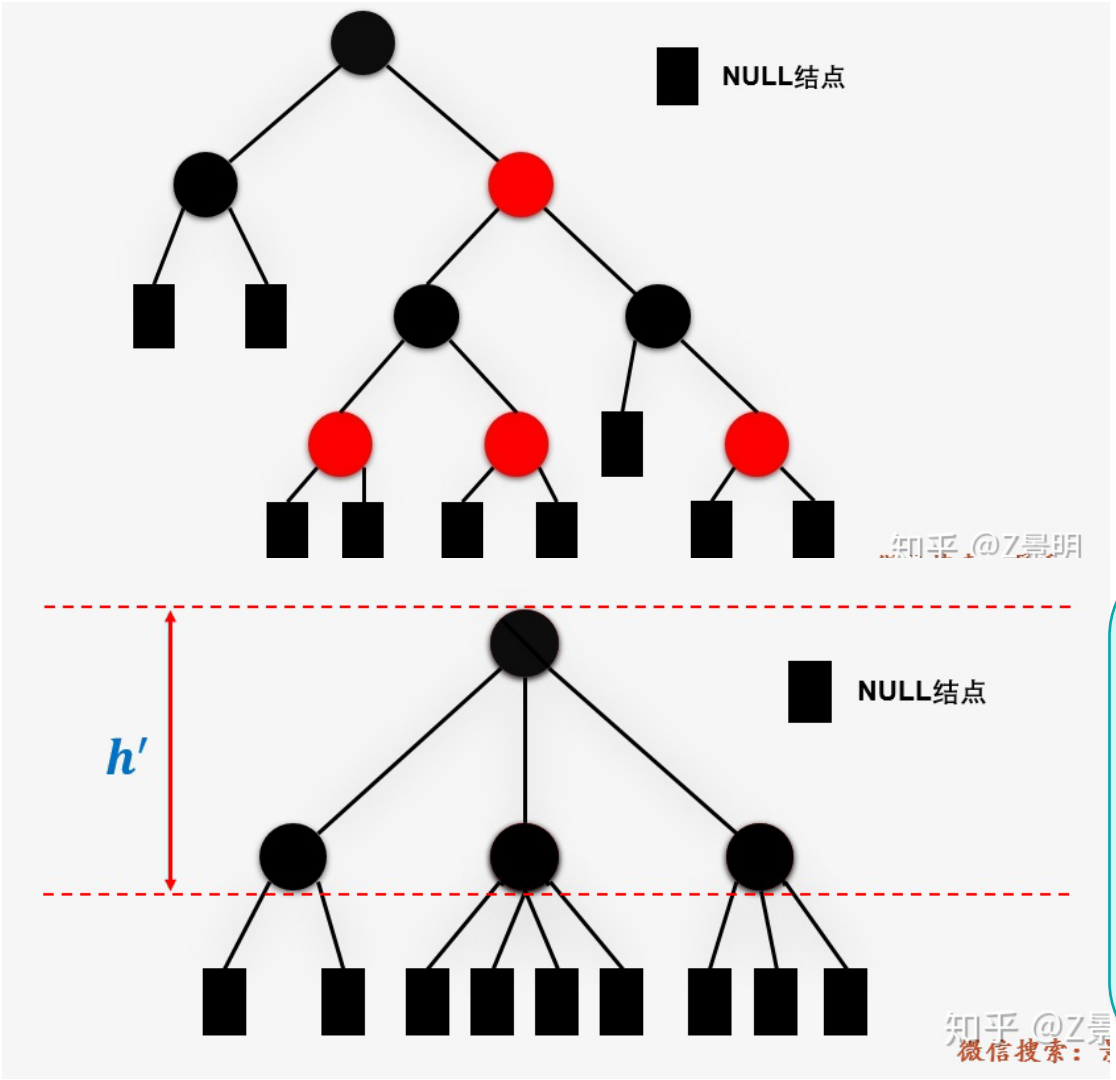
$$bh \geq \frac{h}{2}$$

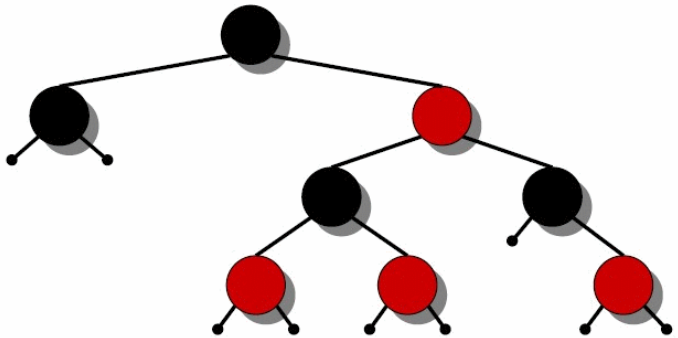
The number of nodes from the root node to the farthest leaf node is called **tree height h**. Since in a red-black tree, the children of a red node must be black nodes, the number of red is less than the number of black nodes and must be less than half.

PS The approximate balance of the red-black trees

3) Lemma: The height of a red-black tree with n internal nodes $h \leq 2\log(n+1)$

A. Merge all red nodes into their black parent nodes





- ✓ Merging produces a 2-3-4 tree
- ✓ Each node in this tree has 2, 3 or 4 child nodes
- ✓ The leaf nodes of a 2-3-4 tree have the same depth h'

PS **The approximate balance of the red-black trees**

◆ **利用红黑树的Treemap.tailMap(key)方法，找到比key大的map集合**

3) Lemma: The height of a red-black tree with n internal nodes $h \leq 2\log(n+1)$

* The number of red nodes is at most half of the path height h from the root node to the leaf node

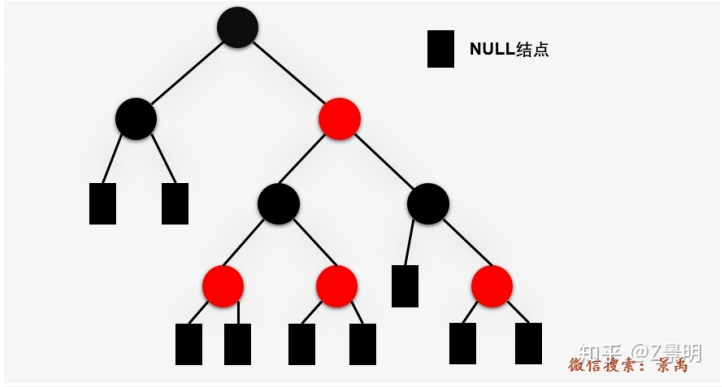
For a 2-3-4 tree with n+1 NULL nodes, the following conclusions:

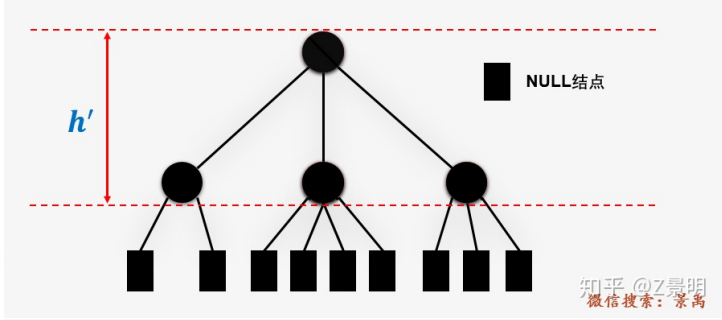
1. $n+1 \geq 2^{h'}$
2. $\log(n+1) \geq h' \geq h/2$
3. $h \leq 2\log(n+1)$

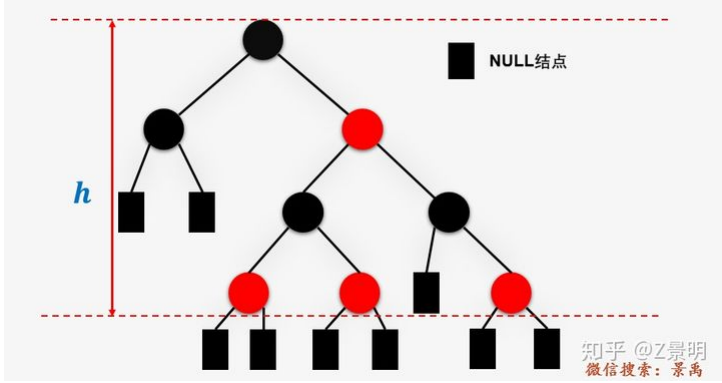
So for a red-black tree with n nodes, the time complexity is **O(logn)** regardless of finding, deleting, finding the maximum value, minimum value, etc.

A. Most self-balancing BST library functions are implemented with red-black trees, such as **map** and **set** in C++

B. Red-black trees are also used to implement CPU scheduling in Linux operating systems.



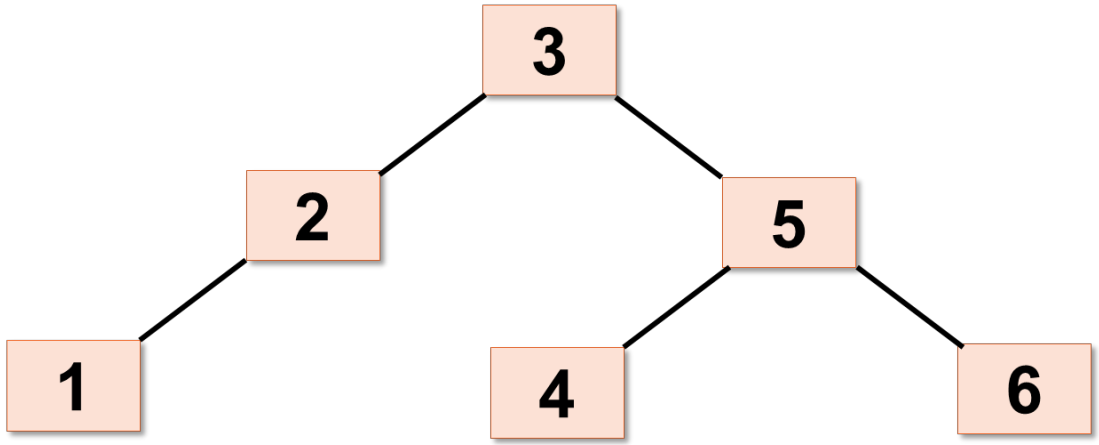




PS Balanced Binary Tree (AVL)

Characteristics

1) the difference in height between the left and right subtrees does not exceed 1;
2) It can be an empty tree;
3) If it is not an empty tree, the left subtree and the right subtree of any node are balanced binary trees and the absolute value of the difference in height does not exceed 1.



平衡二叉树AVL

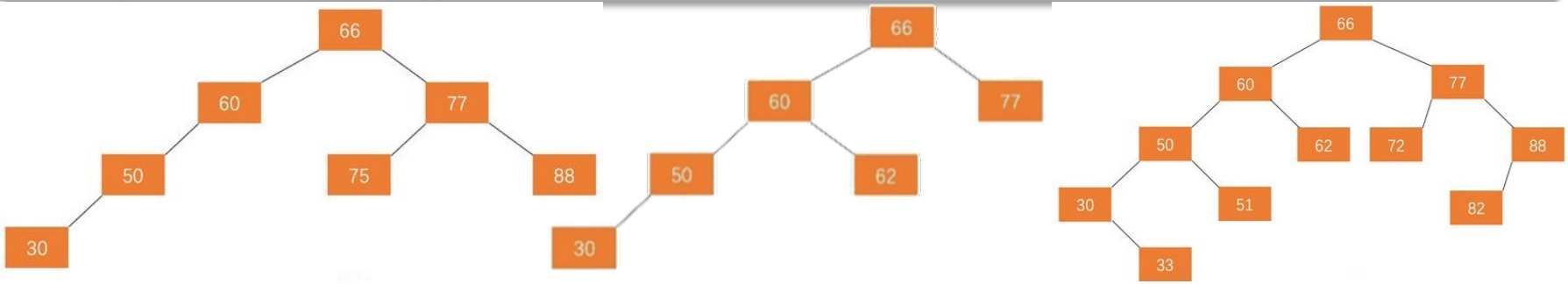
When the number of nodes is fixed and **kept the left and right subtree balanced**, the tree is found most efficiently

PS Balanced Binary Tree (AVL)

When the number of nodes is fixed and **kept the left and right subtree balanced**, the tree is found most efficiently

Balance Factor, BF

The difference in height (depth) between the left and right subtree of a node is the balancing factor of that node. In a balanced binary tree, the balance factor of a node takes only 0, 1 or -1, which corresponds to equal height of the left and right subtrees, respectively, with the left subtree being taller and the right subtree being taller.

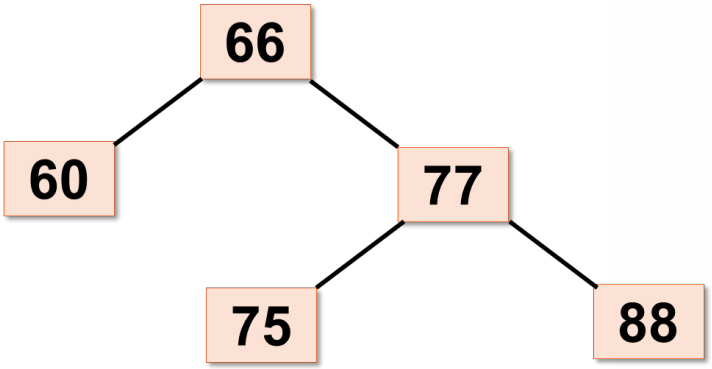


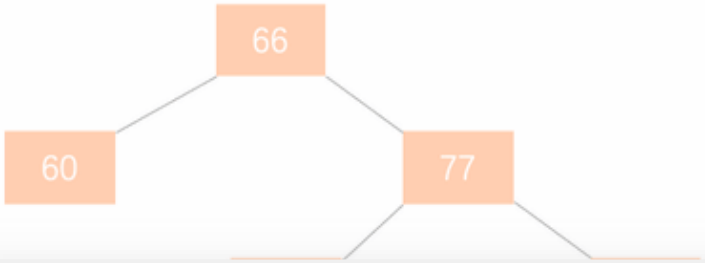
With more than one node in a tree, you can imagine the large number of **imbalance adjustments** that need to be experienced in order to ensure that all nodes are balanced

PS Balanced Binary Tree (AVL)

- Consider not only the time complexity but also the execution complexity

If **|balance factor|>1** after rotation, we still have to **continue to rotate**. Balanced binary tree pursues **absolute balance**, the condition is harsher.





The left subtree of node 66 has a he
factor is -2 and the tree is out of bala
called the **minimum imbalance sub**

The subtree that looks up at the new
absolute value of the first balance fa
subtree. And this time, we can adjust the unbalanced tree to a balanced tree by

- AVL trees have a lot of **rotation** operations during **insertion** and **deletion**
- If the application involves **frequent insertion and deletion** operations, AVL should not be chosen in favor of **red-black trees**.
- If **lookup** operations are relatively more frequent, **AVL** is preferred.

The imbalance adjustment of balanced binary trees is mainly achieved by **rotating the least imbalanced subtree**

1.5 History of Consistent Hashing

A theory appears first, but nobody asks for it

1997 The implementation of consistent hashing given in this lecture first appeared in a research paper in STOC ("Symposium on the Theory of Computing")—this is one of the main conferences in theoretical computer science.10 Ironically, the paper had previously been rejected from a theoretical computer science conference because at least one reviewer felt that "it had no hope of being practical."

金风未动蝉先觉，春江水暖鸭先知—**Some preparations are always brewing secretly.**

1998 Akamai is founded.

好风凭借力，送我上青云——**The theory is combined with the practice, and the advantages are reflected in the practicality.**

1999.3 A trailer for "Star Wars: The Phantom Menace" is released online, with Apple the exclusive official distributor. apple.com goes down almost immediately due to the overwhelming number of download requests. For a good part of the day, the only place to watch (an unauthorized copy?) of the trailer is via Akamai's Web caches. This put Akamai on the map.

1.5 History of Consistent Hashing

小心电话诈骗——**Integrity is needed worldwide, otherwise opportunities may disappear without a second thought**

1999.4 Steve Jobs, having noticed Akamai's performance the day before, calls Akamai's President Paul Sagan to talk. Sagan hangs up on Jobs, thinking it's an April Fool's prank by one of the co-founders, Danny Lewin or Tom Leighton.

The path of development may have various twists and turns.

2001.9 Tragically, co-founder Danny Lewin is killed aboard the first airplane that crashes into the World Trade Center.

实力总会为你赢来下一个机会——**Please study hard, build up your strength and wait for the flowers to bloom**

2001 Consistent hashing is re-purposed to address technical challenges that arise in peer-to-peer (P2P) networks. **A key issue** in P2P networks is how to keep track of where to look for a file, such as an mp3. This functionality is often called a "distributed hash table (DHT)." DHTs were a very hot topic of research in the early years of the 21st century.

1.5 History of Consistent Hashing

实力总会为你赢来下一个机会——Please study hard, build up your strength and wait for the flowers to bloom

2001 Consistent hashing is re-purposed to address technical challenges that arise in peer-to-peer (P2P) networks. **A key issue** in P2P networks is how to keep track of where to look for a file, such as an mp3. This functionality is often called a "distributed hash table (DHT)." DHTs were a very hot topic of research in the early years of the 21st century.

First-generation P2P networks: **centralized topology network**-solved this problem by having a centralized server keep track of where everything is. Such a network has a single point of failure, and thus is also easy to shut down. **Napster结构**

Second-generation P2P networks: **Fully Distributed Unstructured Topology Networks**-used broadcasting protocols so that everyone could keep track of where everything is. This is an expensive solution that does not scale well with the number of nodes. **Gnutella结构**

Third-generation P2P networks: **fully distributed structured topology**-use consistent hashing to keep track of what's where. Consistent hashing remains in use in modern P2P networks, including for some features of the BitTorrent protocol. **chord项目**

1.5 History of Consistent Hashing

最好的技术总会蔓延开——To be a blacksmith, you need to be tough yourself

2006 Amazon implements its internal Dynamo system using consistent hashing. The goal of this system is to store tons of stuff using commodity hardware while maintaining a very fast response time. As much data as possible is stored in main memory, and consistent hashing is used to keep track of what's where.

问题	Dynamo采用的技术
数据分布	改进的一致性哈希（DHT），采用了虚拟节点技术
复制协议	复制写协议（Replicated-write protocal，NRW参数可调）
数据冲突处理	向量时钟
临时故障处理	数据回传机制（Hinted handoff）
永久故障的恢复	Merkle哈希树，简单讲就是二叉树的复制
成员资格及错误检测	基于Gossip的成员资格和错误检测协议

1.6 Bloom Filter

A: Suppose you have built an online teaching system, in which the registration page needs to implement a function that **retrieves whether the user's registered account name exists in the system's database**, and outputs a text reminder immediately if it already exists.

A: **How would you quickly determine if an element (e.g. account name) exists in a collection (list of usernames)?**

B: I will calculate the hashcode of each input username and compare it, if it is the same, I will tell the user to change the name (Time Complexity $O(1)$)

A: Good answer, listen carefully and remember **hash**. But, hash is not the best way to deal with it, because it may be misjudged (don't forget the **hash conflict**), and this operation, just to record the **hashcode will take up a lot of storage space.**

A: Each user name is stored as a hashcode, which takes up a lot of space; not recording it in advance but calculating it when retrieving, which consumes horrible computing resources

B: Is there any way to **reduce the false positive rate** and at the same time **reduce the amount of resources used**?

1.6 Bloom Filter

Bloom Filter

Bloom Filter has a wide range of applications in **de-duplication of big data, and determining whether data exists or not**. It is a space-efficient(空间效率高) probabilistic algorithm and data structure for determining whether an element is in a set, and its core is a very long binary vector and a series of hash functions.

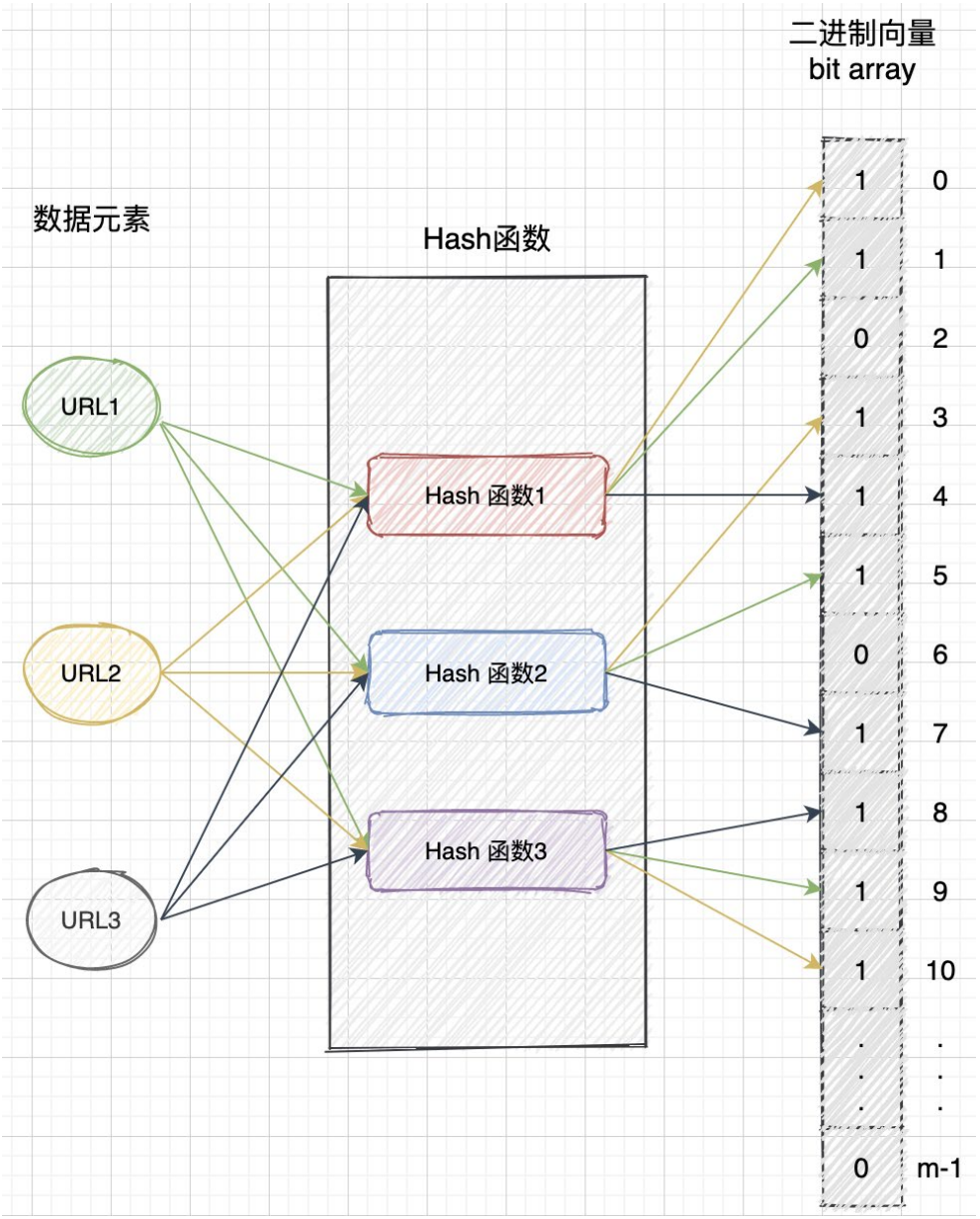
Principle: when an element is added to the set, **the element is mapped to K points in a bit array** by **K hash functions**, and **they are set to 1**. When retrieving, we only need to see **if all these points are 1** to (**approximately**) know whether it is in the set or not: if any of these points is 0, then the checked element **must not be** there; if all of them are 1, then the checked element is likely to be is **likely to be** there.

否定一个人很容易，但是肯定他却很难——原来电脑（布隆滤波器）也会这样

Bloom Filter: I just can't tell if it will be the same name mixed in the set, 不是歧视啊喂！！

1.6 Bloom Filter

Principle: when an element is added to the set, **the element is mapped to K points in a bit array** by **K hash functions**, and **they are set to 1**. When retrieving, we only need to see **if all these points are 1** to (**approximately**) know whether it is in the set or not: if any of these points is 0, then the checked element **must not be** there; if all of them are 1, then the checked element is likely to be is **likely to be** there.

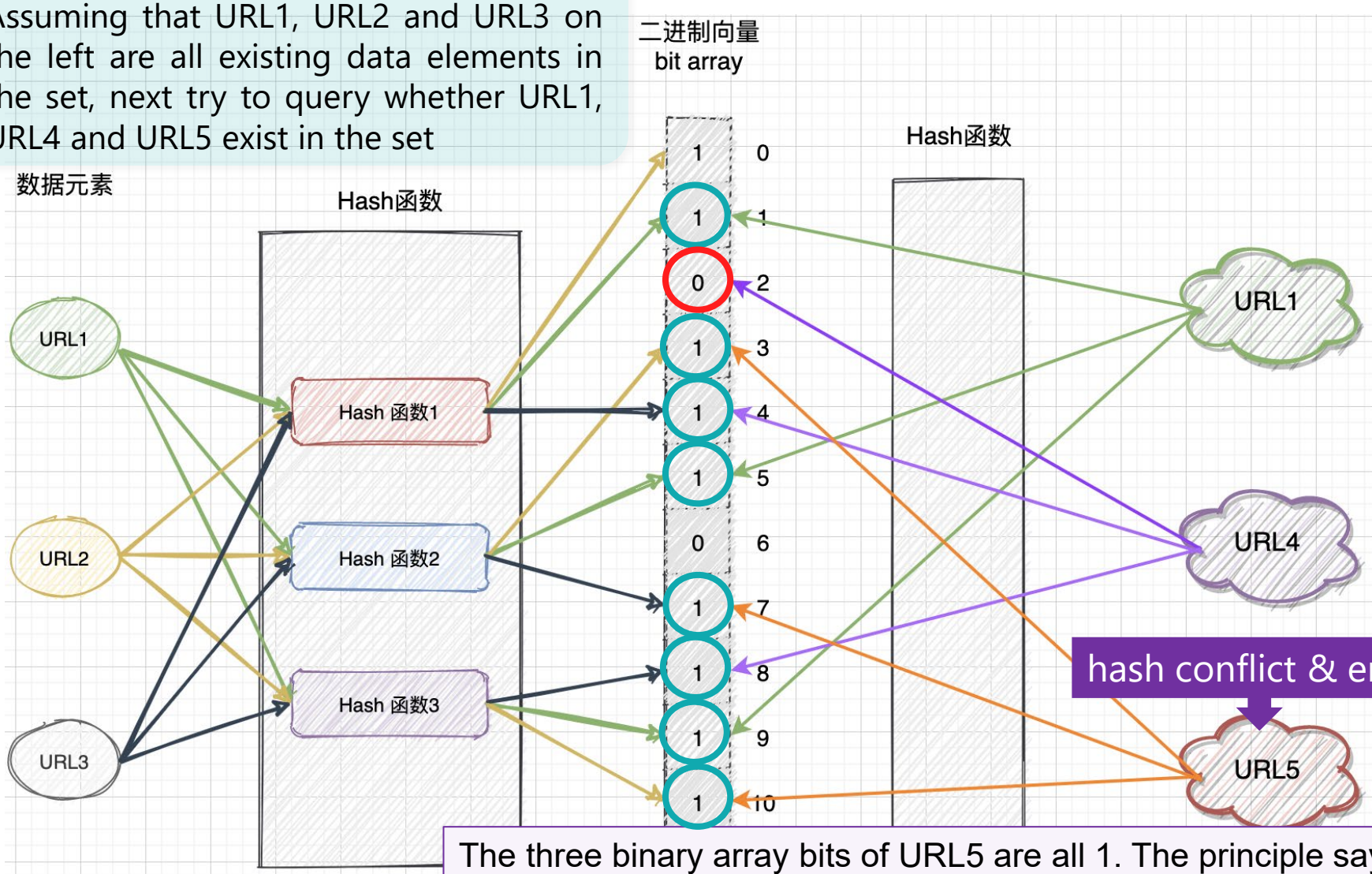


Suppose K=3

- Each data element will go through 3 different hash functions
- correspond to a different binary array of bits and set to 1
- reduces the probability of hash conflict & the error rate

1.6 Bloom Filter

Assuming that URL1, URL2 and URL3 on the left are all existing data elements in the set, next try to query whether URL1, URL4 and URL5 exist in the set



The three binary array bits of URL5 are all 1. The principle says that the result is that the URL5 element exists in BloomFilter, but you can clearly see that the query has an error! This is the case where the **query produces an error**.

Existing

1.6 Advantages and Disadvantages of Bloom Filter

✓ A Bloom filter that says an element is **present may be** a **misjudgment**
× A Bloom filter says an element is not present, then it **must not be present**

Advantage:

1. space saving

Storing a URL (64B) requires 64*8=512bit storage space, while the result of Bloom filtering is stored in a binary array, where an element is hashed once and mapped 1bit. In contrast, Bloom Filter is extremely efficient in terms of space efficiency.

2. high performance

Bloom Filter doesn't need to look through all the elements like a chain table query. Instead, it will find the required result by taking the subscripts directly just like an array. The performance is high.

Disadvantage:

1. Misjudgment

The URL5 query has a miss, which may be because there are too few (or too many) hash functions; or it may be because the binary array is too small and the data is too much and has been reset by too much input data. In engineering applications, we want to keep the false positives below **0.01%**.

2. Can't delete elements

When stored in binary array, the data elements are hashed and the corresponding position is set to 1 (if it has been set to 1 by the element A, the value of the position is no longer modified by the element B). It is impossible to confirm the source of the 1 in the array, so it is absolutely impossible to delete the element and set its corresponding position to 0 (otherwise it is A that you want to delete, and as a result B is also affected).

1.6 Advantages and Disadvantages of Bloom Filter

Probability of false positives

1. Assume that a hash function selects each array position with **equal probability**. If m is the **number of bits in the array**, the probability that **a certain bit is not set to 1 by a certain hash function during the insertion of an element is**

$$1 - \frac{1}{m}$$

One of the m elements of the array has been set to 1

2. If k is the **number of hash functions** and each has no significant correlation between each other, then the probability that **the bit is not set to 1 by any of the hash functions is**

$$\left(1 - \frac{1}{m}\right)^{k}$$

K: superscript, k times set 1 still 0

3. If we have inserted n elements, **the probability that a certain bit is still 0 is**

$$\left(1 - \frac{1}{m}\right)^{kn}$$

4. We can use the well-known identity for e^{-1}

$$\lim_{m\to\infty}\left(1 - \frac{1}{m}\right)^{m} = \frac{1}{e}$$

5. Bringing (4) into (2) (3) gives

$$\left(1 - \frac{1}{m}\right)^{kn} = \left(\left(1 - \frac{1}{m}\right)^{m}\right)^{\frac{k}{m}\cdot n} \approx e^{-kn/m}$$

1.6 Advantages and Disadvantages of Bloom Filter

6. the probability that it is **1** is therefore

$$1-\left(1-\frac{1}{m}\right)^{kn} \approx 1-e^{-kn/m}$$

7. Now **test membership of an element that is not in the set**. Each of the k array positions computed by the hash functions is 1 with a probability as above. The probability of all of them being 1, which would cause the algorithm to erroneously claim that the element is in the set, **False Positives** is often given as

$$\rho=\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)^{k} \approx \left(1-e^{-kn/m}\right)^{k}$$

8. For what value of **k** is the false positive ρ possibility the lowest?

$$k=\ln 2\cdot\frac{m}{n}$$

If $f(k)=\left(1-e^{-kn/m}\right)^{k}$, $b=e^{\frac{n}{m}} \Rightarrow f(k)=\left(1-b^{-k}\right)^{k}$

$\Rightarrow \ln f(k)=k\ln\left(1-b^{-k}\right)$, **求导(寻找极值常见方法)**

$$\Rightarrow \frac{1}{f(k)}\cdot f'(k)=\ln\left(1-b^{-k}\right)+k\cdot\frac{1}{1-b^{-k}}\cdot\left(-b^{-k}\right)\cdot\ln b\cdot(-1)$$

$$=\ln\left(1-b^{-k}\right)+k\cdot\frac{b^{-k}\cdot\ln b}{1-b^{-k}}$$

If $f'(k)=0$, $\ln\left(1-b^{-k}\right)+k\cdot\frac{b^{-k}\cdot\ln b}{1-b^{-k}}=0 \Rightarrow \left(1-b^{-k}\right)\cdot\ln\left(1-b^{-k}\right)=-k\cdot b^{-k}\cdot\ln b=b^{-k}\cdot\ln b^{-k}$

$\Rightarrow 1-b^{-k}=b^{-k} \Rightarrow b^{-k}=1/2 \Rightarrow e^{-\frac{kn}{m}}=\frac{1}{2} \Rightarrow k=\ln 2\cdot\frac{m}{n}$

1.6 Advantages and Disadvantages of Bloom Filter

8. The **required number of bits, m, given n** (the number of inserted elements) and a desired false positive probability ρ (lowestρ, assuming the optimal value of k is used) can be computed by substituting the **optimal value of k** in the probability expression **(7)** above:

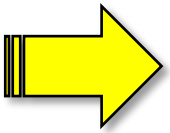
$$\rho = \left(1 - e^{-kn/m}\right)^k,\ k = \ln 2 \cdot \frac{m}{n} \Rightarrow \rho = \left(1 - e^{-\frac{n}{m}\left(\frac{m}{n}\ln 2\right)}\right)^{\frac{m}{n}\ln 2}$$

$$\Rightarrow \ln \rho = \frac{m}{n}\ln 2\left(\ln \frac{1}{2}\right) = -\frac{m}{n}(\ln 2)^2$$

$$\Rightarrow m = -\frac{n \ln \rho}{(\ln 2)^2}$$

$$m = -\frac{n \ln p}{(\ln 2)^2}$$

$$k = \ln 2 \cdot \frac{m}{n}$$



If false positive ρ has an exact requirement, the value of k is taken independent of m and n

1.6 Advantages and Disadvantages of Bloom Filter

How large a binary array and how many hash functions are needed to keep the false positives **below 0.01%** in the above case?

1. Selection of the length m of the binary bit array

$$m = -\frac{n \ln p}{(\ln 2)^2}$$

n: Number of data elements, p: **0.01%** $m = 19.19n$

We can get: **m=19.19n**, that is, the length of the binary bit array is about 20 times the number of elements n. If there are 10 billion pieces of data, the binary array needs 20 billion bits, which is equivalent to about 25GB of space

2. How many hash functions K are preferably needed

$$K = \ln 2 \times \frac{m}{n} \approx 0.7 \times \frac{m}{n} \approx 14$$

Generally using this formula, k hash functions are needed, and according to the given false positive rate, **14 hash functions** are needed for the construction

1.7 Heavy Hitters

Q: Given a massive stream of access logs (e.g., at a daily scale of hundreds of billions), how can we **quickly** and a**pproximately** (without requiring 100% precision) statistically determine the **most frequently** accessed domains and their access **frequencies**?.

Identifying the **top k most frequent items** in a data stream (also known as the **Heavy Hitters** problem).

Approach 1: HashMap + Min-Heap

Scenario: Identifying the top k frequent items in a data stream (e.g., hot search keywords, high-frequency IPs).

Example: *Counting top-liked comments on Bilibili*.

Premise

1. Frequency Counting with HashMap

Use a HashMap<String, Long> to track the frequency of each element.

Example: *Record the like counts of all comments under a specific video using the HashMap*.

1.7 HashMap + Min-Heap

2. Top-K Maintenance with Min-Heap

Maintain a Min-Heap with capacity k. The heap's top element is the minimum value in the current Top-K set.

Example: *Somebody want to award prizes to the top 10 comments with the highest like counts. So, he set the heap capacity to 10.*

Due to the Min-Heap property (parent nodes ≤ child nodes), the 10th-ranked comment by likes resides at the heap top.

Any new comment with a higher like count than the heap top replaces it (since all other heap elements are larger, representing the top 1-9 ranked comments, which are less likely to be displaced).

Dynamic Updates

1. **Update Frequency in HashMap**

For each element retrieved from the data stream:

- If the element exists in the HashMap, increment its count by 1.
- If the element does not exist, insert it into the HashMap with an initial count of 1.

Example: *When a comment receives a new like, it is added to the HashMap (if new) or its existing like count is incremented.*

2. Update the Heap

- **If the element exists in the heap:**
- Increment its count in the heap.
- Adjust the heap (e.g., sift up/down to restore the Min-Heap property).

- **If the element is not in the heap:**
- Compare its count with the heap top element:
- If the element's count > heap top's count:
- Replace the heap top with this element.
- Adjust the heap (e.g., sift down to maintain heap structure).

Example:

If an existing highly-liked comment gains new likes, increment its count in the Min-Heap and trigger heap adjustments if necessary.

If a new comment's like count exceeds the heap top (the current 10th-ranked entry), it replaces the top element, followed by heap adjustments to ensure consistency.

Space Complexity:

- The **HashMap** requires storing all elements, resulting in $O(n)$ space.
- The **heap** needs to store k elements, requiring $O(k)$ space.

Since $O(k)$ is negligible compared to $O(n)$ (assuming $k \ll n$), the total space complexity remains $\boldsymbol{O(n)}$

Time Complexity

1. HashMap $\boldsymbol{O(1)}$:

Each element in a HashMap is a **Node** containing a **key**, a **value**, and a pointer to the next node.

- **Lookup Operations:**
  - Best Case: $O(1)$ — Achieved with a uniformly distributed hash function and no collisions.
  - Worst Case: $O(n)$ — Occurs when searching a bucket with the longest linked list, where n is the length of that list.

- **Insertion Operations:**
  - Best Case: $O(1)$ — Directly placing the element in an empty bucket.
  - Worst Case: $O(n)$ — If all insertions collide and append to the same linked list.
- **Optimization:**

When a linked list exceeds a threshold (e.g., Java 8+), it is converted to a **red-black tree**, reducing worst-case lookup, insertion, and deletion time to $O(\log n)$.

2. Heap $O(k + \log k)$

- As shown in the video, the time complexity for **inserting** a new element into the heap or **extracting the minimum element** (in a Min-Heap) is $O(\log k)$.
- However, **searching for a specific element** in a Min-Heap or Max-Heap has a time complexity of $O(k)$, *as heaps are not optimized for search operations (they prioritize fast insertions and deletions, typical of priority queues)*.
- Thus, the combined time complexity for **searching an element + adjusting the heap** is $O(k + \log k)$.

3. Summary $O(n)$:

•For each new element processed, the time complexity is $O(1 + k + \log k)$.

•For n elements, the total time complexity becomes $O(n(1 + k + \log k))$.

•Since k is a constant (e.g., fixed Top-K size), this simplifies to $O(n)$.

Q: If the dataset is too large to fit in a single machine's memory, how to handle this?

Two solutions are proposed:
Approach 2: Multi-Machine HashMap + Heap & Approach 3: Count-Min Sketch + Heap

Approach 2: Multi-Machine HashMap + Heap

Data Sharding: Partition the data stream across machines using a hash function.

Example: *With 8 machines:*

Machine 1 processes elements where $hash(elem)\ \%\ 8\ ==\ 0.$

Machine 2 processes elements where $hash(elem)\ \%\ 8\ ==\ 1.$

... and so on.

Local Processing:

Each machine maintains its own **HashMap** (for frequency counting) and **Min-Heap** (for local Top-k tracking).

Each machine independently computes its local Top-k elements.

Global Aggregation:

Collect all local Heaps from the machines over the network.

Merge the Heaps into a single global Heap to determine the **final Top-k elements**.

1.8 Count-Min Sketch(计数-最小草图)

The **Count-Min Sketch** is a probabilistic data structure designed to efficiently estimate the frequency of elements in a data stream. Its core concept utilizes **multiple hash functions and a two-dimensional counter array**, trading space for time to **approximate** the counts of **high-frequency elements**. 【VS Bloom Filter】

1. Implementation Steps

1.1 Initialization Parameters

Width: ***e.g.*** $w=5$

- Defines the number of columns w in the counter array.
- Typically correlated with the **desired error bound** (larger width reduces estimation error).

Depth: ***e.g.*** $d = 3$

Defines the number of rows (i.e., the number of hash functions).

A greater depth improves accuracy (reduces hash collision probability).

Hash Function Set:

Select d pairwise independent hash functions (e.g., MurmurHash, Fowler-Noll-Vo).

$$d\begin{cases} h_1(x) = \mathbf{ASCII}(x) \\ h_2(x) = 2 + \mathbf{ASCII}(x) \\ h_2(x) = 4 \cdot \mathbf{ASCII}(x) \end{cases}$$

1.2 Data Structure

Create a two-dimensional integer **array counters**$[d][w]$, initialized to all 0.

	0	1	2	3	4
$h1(x) = \mathbf{ASCII}(x)$	0	0	0	0	0
$h2(x) = 2 + \mathbf{ASCII}(x)$	0	0	0	0	0
$h3(x) = 4 \cdot \mathbf{ASCII}(x)$	0	0	0	0	0

	0	1	2	3	4
$h1(x) = \mathbf{ASCII}(x)$	0	**1**	0	0	0
$h2(x) = 2 + \mathbf{ASCII}(x)$	0	0	0	**1**	0
$h3(x) = 4 \cdot \mathbf{ASCII}(x)$	0	0	0	0	**1**

	0	1	2	3	4
$h1(x) = \mathbf{ASCII}(x)$	**1**	0	0	0	0
$h2(x) = 2 + \mathbf{ASCII}(x)$	0	0	**1**	0	0
$h3(x) = 4 \cdot \mathbf{ASCII}(x)$	**1**	0	0	0	0

1.3 Hash Function Generation

Generate **independent hash functions** for each row, ensuring uniform distribution of **hash values**.

Note: All hash functions must map their outputs to the range [0, w-1] (via modulo w).

1.4 Insert Element (Update Counters) *e.g. add B, ASCII=66 [1,3,4]*

For each input element:

- Compute the column index for each row using the corresponding hash function.
- **Increment** the counter at each computed [row][column] position.

1.5 Query Element Frequency *e.g. find A, ASCII=65 [0,2,0], appear at most 0 times. But for G, ASCII=71 [1,3,4], The corresponding positions are all set to 1, so the letter G can appear at most once (wrong).* **【upper-bound estimate】**

Take the **minimum value** across all counters associated with the element's hash positions. **Why the minimum?** Mitigates overestimation caused by hash collisions (guarantees frequency ≤ true count).

2. Parameter Configuration & Error Control

- **Error Bound (ε):** The **maximum error** in frequency estimation is $\varepsilon \times N$, where N is the total number of elements in the data stream.
- **Confidence Probability $(1 - \delta)$:** The result is correct with a probability of at least $1 - \delta$.
- **Interpretation:** across all elements
- **Parameter Formulas:**
  - **Width:** $w = \left\lceil \frac{e}{\varepsilon} \right\rceil$, **Depth:** $d = \left\lceil \ln\left(\frac{1}{\delta}\right) \right\rceil$ **(ceiling 上取整)**
  - e is the base of the natural logarithm (approximately equal to 2.718)

For $\varepsilon = 0.01$ and $\delta = 0.05$:

- $w = \lceil e/0.01 \rceil \approx 272 \rightarrow$ Simplified to **width = 200** (let $e \approx 2$).
- $d = \lceil \ln(20) \rceil = \lceil 2.996 \rceil = 3$.

- **Requirement**: Count frequencies in a data stream of size $N = 10^6$.
- **Error Tolerance**: Set $\varepsilon \times N = 2000 \rightarrow \ \varepsilon = 0.002$.
- **Width Calculation**: $w = \lceil e/0.002 \rceil \approx \lceil 1359.14 \rceil = 1360$.
- **Confidence**: For 99% confidence ($\delta = 0.01$):

$d = \lceil \ln(100) \rceil = \lceil 4.605 \rceil = 5$.

Memory Footprint:

- Each counter occupies **4 Bytes**.
- Total memory: $w \times d \times sizeof(counter) = 1360 \times 5 \times 4Bytes \approx 27.2KB$

Error Model and Single-Row Analysis 误差模型与单行分析

•**Goal:** Ensure that the error between the estimated value $\hat{f}_x$ and the true frequency f_x does not exceed $\varepsilon \times N$ (where N is the total number of elements in the data stream), with probability at least $1 - \delta$.

- $\boldsymbol{\varepsilon}$: Upper bound of the allowed error. The error bound for N elements is $\varepsilon \times N$
- $\boldsymbol{\delta}$: Probability that the estimated value exceeds the error bound
- **Confidence Probability $1 - \delta$:** The probability that the result is correct (error does not exceed the bound) is at least $1-\delta$
- $\boldsymbol{w}$**:** Number of columns in the Count-Min Sketch (i.e., the size of the hash table)
- $\boldsymbol{d}$**:** Number of rows in the Count-Min Sketch (i.e., the number of independent hash functions)

The core idea of the Count-Min Sketch is to map data into a two-dimensional array using multiple hash functions, and then estimate the frequency of an element by taking the minimum of the corresponding counters. Due to hash collisions, the estimate may **overestimate the true frequency**. To control this overestimation, we need to ensure that the increment to each counter is **small enough** so that the **total error** remains within an acceptable range.

Derivation Process:

Assume each counter is incremented by 1.

For an element with true frequency f_x ,its estimated frequency in the Count-Min Sketch is $\hat{f}_x$.

Due to hash collisions, $\hat{f}_x$ may overestimate f_x ,i.e., $\hat{f}_x \geq f_x$.

We desire $\hat{f}_x \leq f_x + \varepsilon \times N$.

To guarantee this, the width w of the Count-Min Sketch must be large enough so that the increments to each counter are spread across multiple positions. Specifically, the choice of w must satisfy the following condition:

Single-row error analysis:

The probability that an element is hashed into a given bucket is $1/w$.

The total number of events is N, so the expected count per bucket is $\mu = N/w$.

Applying **Markov's inequality** to bound the probability that the single-row error exceeds $\varepsilon \times N$:

$$\boldsymbol{Pr}(\textbf{Single-row error 单行误差} \geq \boldsymbol{\varepsilon N}) \leq \frac{\boldsymbol{\mu}}{\boldsymbol{\varepsilon N}} = \frac{\mathbf{1}}{\boldsymbol{\varepsilon \cdot w}}$$

We require this probability to be at most $1/e$ (to balance computational efficiency and error):

$$\frac{1}{\varepsilon \cdot w} \leq \frac{1}{e} \implies w \geq \frac{e}{\varepsilon}.$$

Thus, we set **width $= e/\varepsilon$.**

Markov's Inequality is a fundamental tool in probability theory used to bound the probability that a non-negative random variable exceeds a given threshold. In the Count-Min Sketch, it is used to control the probability that the error in a single hash table row exceeds $\varepsilon \times N$.

Definition of Markov's Inequality：

For a non-negative random variable $X \geq 0$ and any constant $a > 0$, Markov's Inequality:

$$\mathbf{Pr}(\boldsymbol{X} \geq \boldsymbol{a}) \leq \frac{\boldsymbol{E}[\boldsymbol{X}]}{\boldsymbol{a}}$$

That is, the probability that X exceeds a is at most the ratio of its expected value $\boldsymbol{E}[\boldsymbol{X}]$ to a.

- **Application in Count-Min Sketch:**

1. Single-row error model:

Goal: Analyze the error between the estimated value $\hat{f}_x$ and the true frequency f_x in a single row of the hash table.

Source of error: Hash collisions from other elements may overestimate the count in the current bucket.

Non-negativity: The error is ≥ 0 (because hash collisions only increase the count).

2. Computation of the expected error:

Assume there are N elements in total, and each element is hashed into a particular bucket in a given row with probability $1/w$.

The expected error in a single row (due to collisions from all other $\boldsymbol{N}$ elements) is:

$$\boldsymbol{E}[\boldsymbol{error}] = \boldsymbol{N}/\boldsymbol{w}$$

3. Applying Markov's Inequality:

- We want to bound the probability that the error exceeds $\varepsilon \times N$:

$$\mathbf{Pr}(\boldsymbol{error} \geq \boldsymbol{\varepsilon N}) \leq \frac{\boldsymbol{E}[\text{error}]}{\boldsymbol{\varepsilon N}} = \frac{\boldsymbol{N/w}}{\boldsymbol{\varepsilon N}} = \frac{\mathbf{1}}{\boldsymbol{\varepsilon \cdot w}}$$

Parameter setting:

To ensure that this probability does not exceed $1/e$ (as required for the **Multi-row joint probability analysis**), we set:

$$\frac{\mathbf{1}}{\boldsymbol{\varepsilon \cdot w}} \leq \frac{\mathbf{1}}{\boldsymbol{e}} \Longrightarrow \boldsymbol{w} \geq \frac{\boldsymbol{e}}{\boldsymbol{\varepsilon}}$$

Therefore, the width is set to $\boldsymbol{w} = \frac{\boldsymbol{e}}{\boldsymbol{\varepsilon}}$

- **Bound on the error probability:**
  - Markov's Inequality provides a loose but universally applicable upper bound on the probability.
  - It does not rely on the specific form of the distribution (only requiring non-negativity and the expected value), making it suitable for the general setting of Count-Min Sketch.
- **Trade-off in parameter design:**
  - The larger the width w, the lower the probability of hash collisions, and the smaller the expected error $E[\text{error}]$.
  - By setting $\boldsymbol{w} = \boldsymbol{e}/\boldsymbol{\varepsilon}$, we control the single-row error probability to $\mathbf{1}/\boldsymbol{e}$ while minimizing space usage.

Multi-row joint probability control:

- **Role of depth: The depth d reduces the joint failure probability by using d independent hash functions.**
- If the error probability of a single row is $1/e$, then the probability that all d rows fail simultaneously is:

$$\left(\frac{1}{e}\right)^{d}$$

- We require the total error probability not to exceed δ:

$$\left(\frac{1}{e}\right)^{d} \leq \delta$$

Using the power rule for logarithms :$\ln\left(a^{b}\right) = b \ln a$, so $d \ln(e^{-1}) \leq \ln \delta$

$$-d \leq \ln \delta \Longrightarrow d \geq -\ln \delta \Longrightarrow \boldsymbol{d \geq \ln\left(\frac{1}{\delta}\right)}$$

Therefore, the depth is set to:

$$\boldsymbol{d = \left\lceil ln\left(\frac{1}{\delta}\right)\right\rceil} \textbf{ (ceiling)}$$

在Count-Min Sketch的参数设置中，选择将单行误差超过εN的概率控制为$1/e$，主要是为了通过后续的联合概率分析简化推导，并优化空间与概率的平衡。

1. Mathematical Simplicity

•**Exponential form of joint probability**:

If the single-row failure probability is set to $p = \frac{1}{e}$,then the total failure probability is $p^d = \left(\frac{1}{e}\right)^d$.

Requiring the total failure probability $\leq \delta$ gives:

$$d \geq \ln\left(\frac{1}{\delta}\right),$$

•with no extra coefficients — the form is extremely concise.

•If another value of p is chosen (e.g. $p = \frac{1}{2}$), the required depth becomes: $d \geq \frac{\ln(1/\delta)}{\ln(1/p)}$

which increases the complexity of the expression.

•**Natural alignment with natural logarithms**:

The natural logarithm and the exponential function e^x are closely linked in calculus, making operations with base e easier to handle.

2. Balancing Space and Probability

Trade-off between width and depth:

When the single-row failure probability is $p = \frac{1}{e}$,the width is $\frac{e}{\varepsilon}$.

If a smaller p is chosen (e.g., $p = \frac{1}{10}$) the width must increase to $\frac{10}{\varepsilon}$,significantly increasing space usage.

If a larger p is chosen (e.g., $p = \frac{1}{2}$,(the depth must increase substantially to maintain the overall failure probability δ, leading to more hash functions and higher computational overhead.

$1/e$ is the balancing point: it achieves the optimal trade-off between controlling space (width) and computational cost (depth).

3. Looseness and Practicality of the Probability Bound

Conservativeness of Markov's inequality:

The probability upper bound given by Markov's inequality is usually loose (the actual error probability may be far below $1/e$).

Choosing $1/e$ as the theoretical value ensures a safety margin while avoiding the complexity of over-optimization.

Acceptability in engineering practice:

In scenarios such as stream processing, a failure probability of $1/e$ is acceptable. Moreover, by adding a small number of depth d, the total failure probability can be reduced exponentially (e.g., when $d = 5$, the total probability is $e^{-5} \approx 0.67\%$.

4. 历史与理论习惯

- **常见于概率算法设计**：
  类似 Bloom Filter 等数据结构也常用自然对数处理独立事件的联合概率，因 e 在概率论中具有普适性。
- **原始论文的选择**：
  Count-Min Sketch 的原始推导（Cormode & Muthukrishnan, 2005）采用此设定，后续研究延续了这一传统。

总结

选择单行失败概率为 **1/e** 的核心原因在于：

1. **简化联合概率分析**，使深度公式 $d = \ln(1/\delta)$ 无需复杂系数。
2. **平衡空间与计算开销**，避免 width 或 depth 的极端增长。
3. **适配自然对数的数学工具**，保持理论推导的优雅性。
4. **符合工程实践需求**，兼顾安全性与效率。

最终，这一设定使得 Count-Min Sketch 的参数既理论严谨，又实际高效。

4. Historical and Theoretical Convention

- **Common in probabilistic algorithm design**:
  Data structures like Bloom filters also frequently use natural logarithms to handle joint probabilities of independent events, because e is ubiquitous in probability theory.
- **Choice in the original paper**:
  The original derivation of the Count-Min Sketch (Cormode & Muthukrishnan, 2005) adopted this setting, and subsequent research has continued this tradition.

Summary

The core reasons for choosing the single-row failure probability as $1/e$ are:

- **Simplifying joint probability analysis**, so that the depth formula $d = \ln(1/\delta)$ requires no additional coefficients.
- **Balancing space and computational overhead**, avoiding extreme growth in either width or depth.
- **Aligning with the mathematical tools of natural logarithms**, maintaining the elegance of the theoretical derivation.
- **Meeting practical engineering needs**, balancing safety and efficiency.

Ultimately, this choice makes the parameters of the Count-Min Sketch both theoretically sound and practically efficient.

3. Complexity Analysis

- **Space Complexity $O(d \times m)$**: The Count-Min Sketch requires a **2D array** of size $d \times m$ (depth × width).
- **Time Complexity $O(n)$**: The algorithm scans the data stream **once**, making it linear in time.

Advantages	Disadvantages
Memory-efficient: Ideal for large-scale streaming data.	**Poor accuracy for low-frequency elements**: Due to hash collisions caused by the **limited array size** (smaller than raw data scale), estimation errors increase significantly for rarely occurring items.

4. Application Scenarios

- **Network Traffic Monitoring**: Identify high-frequency IP addresses or URL requests.
- **Real-Time Recommendation Systems**: Rapidly detect trending products or content.
- **Natural Language Processing**: Approximate word frequency counting in large-scale text streams.

5. Considerations

- **Hash Collisions**:
  1. Multiple elements may map to the same counter, causing **overestimation**.
  2. Mitigation: Increase *width* (reduces per-bucket collisions) or *depth* (reduces cross-bucket collisions).
- **Memory Optimization**:
  1. Use **Conservative Update**: Only increment the **minimum counter** among all hash positions for an element.
  2. *Why?* Reduces overestimation by avoiding redundant increments.
- **Dynamic Scaling**:
  1. Adjust *width* and *depth* dynamically during data surges (requires **rehashing historical data**).
- **Conclusion**

With the above implementation, the Count-Min Sketch efficiently supports **large-scale frequency estimation** tasks with minimal memory overhead, making it ideal for streaming data analytics.

Approach 3: Count-Min Sketch + Heap

Since the HashMap in Solution 1 is **too large** to fit in memory, we can replace it with the **Count-Min Sketch algorithm**.

- During continuous data stream processing:
  1. **Maintain a standard Count-Min Sketch 2D array** (with parameters *d* rows and *m* columns).
  2. **Maintain a min-heap** with capacity k.
- Procedure for each incoming element:
  - **Update Sketch**: Increment the corresponding counters in the Count-Min Sketch.
  - **Heap Operations**:
  1. **If the element exists in the heap**:
     - Increment its counter in the heap.
     - Adjust the heap structure.
  2. **If the element is not in the heap**:
     - Use the Count-Min Sketch's approximate frequency of the element.
     - Compare it with the frequency of the **top element** in the heap.
     - **If higher than the top element's frequency**:
       - Replace the top element with the new element.
       - Adjust the heap structure.

Complexity Analysis:

- **Space Complexity $O(d \times m)$:**
  - Count-Min Sketch: $O(d \times m)$ (where d = rows, m = columns).
  - Min-heap: $O(k)$ (negligible for small k).
- **Time Complexity $O(n)$:**
  - Per element: $O(1)$ for Sketch updates + $O(k + logk)$ for heap lookups/adjustments.
  - Total: $O(n(1 + k + \log k))$ for *n* elements, when $n \gg k$, → $O(n)$.

Required: submission email: luoxsince1989@qq.com, **Deadline:** Before 2026.5.15

(1) Implement consistent hashing (design and implementation) using a programming language of your choice.

(2) Assume the client node addresses are:

127.0.0.1
113.51.11.5
113.51.15.48
114.48.96.125
115.47.12.35
115.47.12.5
110.0.3.2
116.87.95.62
117.47.10.1

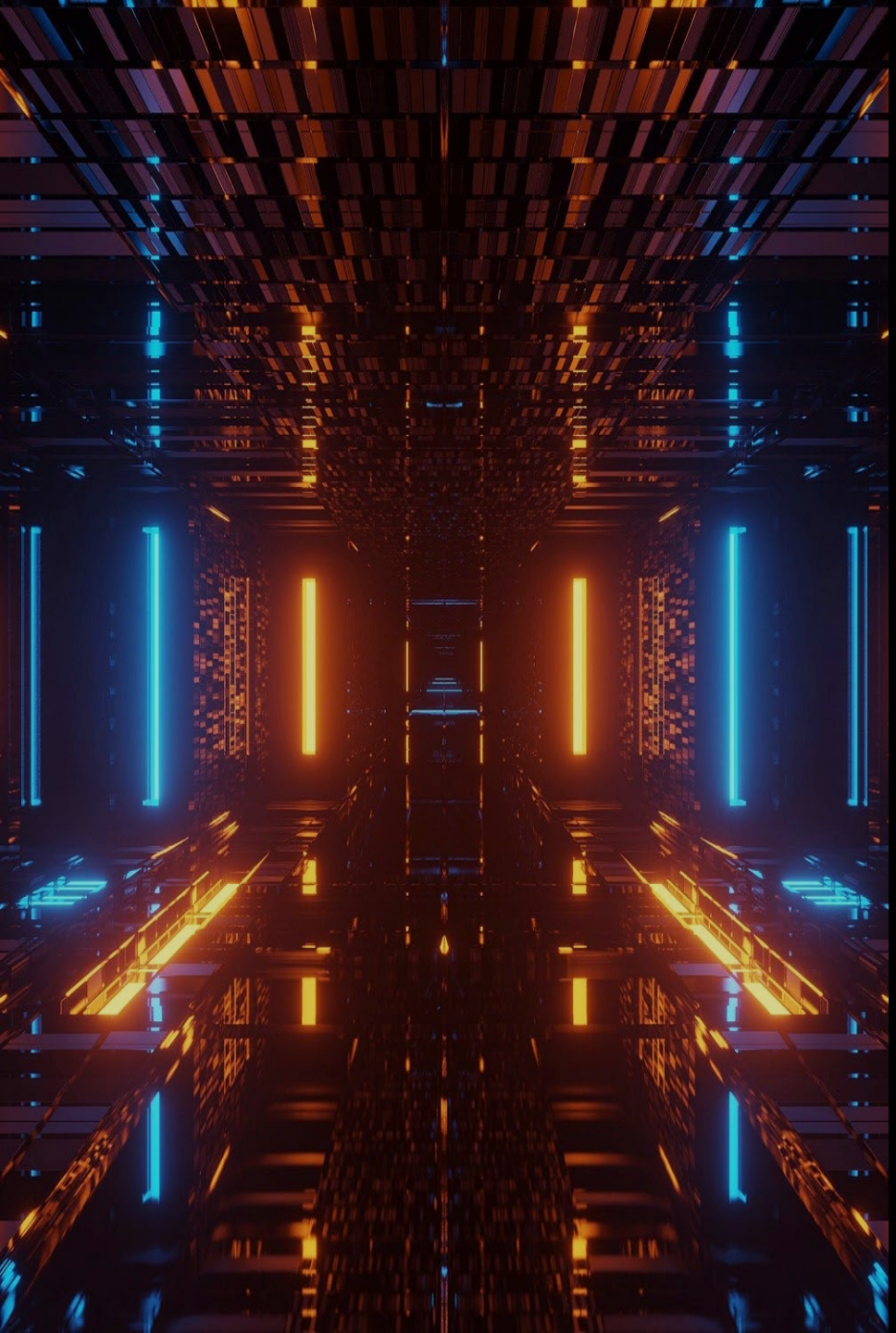
The server node addresses are:

192.168.0.1
192.168.10.2
192.168.20.3
192.168.30.4

Test the mapping relationship between client nodes and server nodes, display the results (including hash values), and answer question (3).

(3) What is the hash function for IP addresses? In your consistent hashing implementation, did you use a sorted tree structure? If so, what is it and at which step was it used?

(4) Evaluate whether there is data skew. If virtual nodes are to be used, how would you design them?



Thanks